MedChat

Graph-Augmented Multimodal RAG System for Oncology

Proof of Concept (POC) — Technical Documentation & Research Log

Organization	Aziro Technologies — Internship Project
Document Type	Full POC Technical Documentation & R&D Log
System Version	v5.1 (cancer_retrieval.py, cancer_ingestion.py, cancer_app.py)
Core Stack	Neo4j Aura · Groq LLaMA-3 · BAAI/bge-base-en · LangChain · Streamlit
Classification	Internal Use Only — Confidential

Table of Contents

Section	Title
1	Executive Summary
2	Problem Statement
3	Limitations of Existing Systems
4	Proposed Solution
5	System Architecture
6	Key Features
7	Tools & Technologies
8	Methodology
8.1	Ingestion Pipeline
8.2	Retrieval Engine
8.3	Answer Generation & Fallback
8.4	Multimodal Processing
8.5	Graph RAG Extension
9	Use Cases
10	Challenges & Solutions
11	Performance Considerations
12	Scalability & Production Readiness
13	Future Scope
14	Weekly R&D Progress
15	Conclusion
16	References

1. Executive Summary

MedChat is a production-grade, graph-augmented Retrieval-Augmented Generation (RAG) system purpose-built for oncology. It was developed as a 16-week POC internship project at Aziro Technologies, evolving from a simple PDF Q&A prototype into a full three-mode clinical assistant backed by a hand-crafted medical knowledge graph in Neo4j Aura.

The system answers clinical questions in three distinct modes: Research mode (grounded in peer-reviewed oncology literature), Graph mode (structured drug–food interactions, nutrition protocols, and treatment side effects from a hand-crafted Neo4j knowledge graph), and Auto mode (both paths fused intelligently). It supports patient report upload, structured analysis, image and figure retrieval from source papers, and real-time web search fallback for out-of-corpus queries.

What Was Built	Why It Matters	Technical Innovation	Status
Graph-augmented clinical RAG assistant with multimodal retrieval and patient report analysis	Cancer patients struggle to interpret medical reports, understand treatment side effects, and find reliable drug–food guidance. Existing chatbots hallucinate or lack clinical depth.	Three-mode routing, dedicated image chunk index, hand-crafted Cypher query library, three-tier web fallback logic	POC Complete — v5.1 operational, RAGAS evaluation framework live

Architecture: The core architectural insight is that different clinical questions require fundamentally different retrieval strategies. Separating these into explicit modes rather than routing opaquely gives users predictable, trustworthy answers and makes failures diagnosable.

2. Problem Statement

Cancer patients undergoing chemotherapy face a convergence of complex, interdependent questions that general-purpose tools cannot answer reliably:

- What foods should I avoid while on cisplatin? Which vegetables interact badly with warfarin when combined with my chemotherapy regimen?
- What does my clinical report actually mean? What is my prognosis given my stage and biomarkers?
- What does Figure 3 in this survival study show? Are there flowcharts explaining my treatment pathway?
- Is there a new FDA-approved drug for my cancer type that my oncologist may not have mentioned?

Existing approaches fail this population in distinct ways:

Failure Mode	Consequence for Cancer Patient
General-purpose LLMs hallucinate clinical facts	A patient receives incorrect drug interaction information — potentially dangerous
Vector-only RAG misses entity-specific queries	"pemetrexed folic acid" returns general nutrition chunks, not the mandatory supplementation protocol
No multimodal retrieval	Questions about Kaplan-Meier curves, PRISMA flowcharts, or treatment decision trees cannot be answered visually
No explicit fallback	Out-of-corpus questions (recent FDA approvals, new trials) are answered with hallucinated "confidence"
No patient report integration	Clinical questions cannot be personalised to the specific patient's diagnosis, stage, or treatment plan

The problem is not simply retrieval quality it is a fundamental mismatch between query types and retrieval strategies. A question about drug–food interactions requires a traversal of a medical knowledge graph with structured relationships. A question about survival statistics requires semantic retrieval from narrative clinical text. A question about a specific figure requires a dedicated visual index. No single retrieval strategy serves all three.

3. Limitations of Existing Systems

3.1 Document Loaders — Comprehensive Evaluation Results

A systematic evaluation of PDF loading libraries was conducted using two test IBM documents , representing different structural complexities. The findings directly drove the choice of Docling + PyMuPDF + RapidOCR as the ingestion backbone.

Library	Tables	Images	Complex PDFs
PyPDF	Fails	Fails	Poor
PDFPlumber	Inaccurate	None	Partial
PyPDFDirectory	Fails	Fails	Poor
PyPDFium2	Fails	Partial	Mixed
PyMuPDF	Basic	Inconsistent	Mixed
PyMuPDF4LLM	Improved	+RapidOCR	Better
PDFMiner	Unreliable	Mediocre	Poor
Docling (standalone)	Excellent	No image desc	Best
Docling + PyMuPDF + RapidOCR	Excellent	Good	Best

3.2 URL & Non-PDF Loaders

Loader	Type	Observations	Decision
WebBaseLoader	URL	Full content retrieved; plain text only, no structure	Used for basic web fallback content
RecursiveUrlLoader	URL	Returns raw HTML; needs custom parsing function for LLM consumption	Viable with post-processing
SiteMapLoader	URL	Could not be configured correctly during testing	Rejected
Docling (URL)	URL	Well-structured, LLM-ready output; accurate scraping with good formatting	Preferred for web ingestion
UnstructuredPPTX	PPTX	Text extracted well; no image extraction support	Used for text-only slides
CSVLoader	CSV	Loads as-is, preserving original structure	Standard use for structured data

3.3 Retrieval Strategy Comparison

This comparison table was derived from hands-on testing during the RAG Search Lab week (Week 5). It directly motivated the Hybrid BM25 + Dense + RRF + MMR architecture.

Test Case	BM25 (Keyword)	Semantic (Vector)	Hybrid	Winner
Specific code: "Error 503"	Excellent: exact match	Poor: matches "Error" generally	Best: exact match prioritised	Hybrid
Synonyms: "login assistance"	Poor: misses "authentication guide"	Excellent: same intent	Balanced: finds both	Hybrid
Short query: "passcode"	Strong: finds every instance	Vague: too much context noise	Strong + filtered for relevance	Hybrid
Long questions	Noisy: distracted by stop words	Strong: focuses on core intent	Best: filters noise, keeps keywords	Hybrid
Drug name: "pemetrexed"	Excellent: exact match critical	Moderate: semantic drift risk	Best: exact name preserved at top	Hybrid

3.4 Retrieval Method Coverage

A comprehensive audit of LangChain-supported retrieval methods was conducted to understand what is natively available versus what required custom implementation.

Retrieval Method	LangChain Support	Status in MedChat
Similarity Search	Native	Used via Neo4jVector
MMR (Maximal Marginal Relevance)	Native	Used post-RRF reranking
BM25	Native (BM25Retriever)	Used for lexical retrieval
Ensemble Retriever	Native	Base pattern; extended to RRF
MultiQuery Retriever	Native	Not used — adds Groq API cost
Contextual Compression	Native	Considered; not implemented yet
Graph Retrieval	Custom required	Implemented via Neo4j Cypher + GraphRetriever class

4. Proposed Solution

MedChat addresses the retrieval mismatch problem with a three-mode architecture where each mode is an explicitly routed pipeline optimised for a specific class of clinical query.

Mode	Routing Trigger	Retrieval Path
RESEARCH	User selects "Research & Literature" in sidebar	BM25 + Dense → RRF → MMR → PDF chunks only
GRAPH	User selects "Treatment & Nutrition" in sidebar	Neo4j Cypher (6 intent templates) first → small vector enrichment appended
AUTO	Default / "Auto" selected; also fires on entity detection	Graph fires if medical entities detected in query; then BM25 + Dense + RRF + MMR merged with graph context

In all three modes, two additional parallel processes run:

- Image chunk retrieval: A dedicated BM25 index of image-only chunks runs independently, injecting visual references into context without competing with content chunks in MMR
- Out-of-corpus detection: Proactive pattern matching triggers web search (DuckDuckGo) before any retrieval for queries about recent FDA approvals, new clinical trials, vaccine approvals, and similar time-sensitive regulatory information

Key Finding: The core innovation is not any single retrieval technique, it is the explicit routing layer that matches each query type to the retrieval strategy most suited to answer it accurately and safely.

5. System Architecture

5.1 Four-Tier Architecture Overview

The system is organised into four tiers that interact sequentially and in parallel depending on query type. The architecture was finalised in v5.1 after 13 weeks of iterative development.

TIER 1 — INGESTION

- PDF → Docling (primary) or PyMuPDF (fallback) → raw text extraction
- 12-step text cleaning: Unicode repair, ligature fixes, header/footer removal, reference section cutoff.
- Caption & image extraction: Docling PictureItem/TableItem → 7-layer noise filter (size, aspect, colour, pHash, publisher pattern)
- Two-stage chunking: MarkdownHeaderTextSplitter → RecursiveCharacterTextSplitter (chunk_size=1200, overlap=200)
- [IMAGE:] tag injection: "## Extracted Visual Assets Database" section appended to markdown before chunking
- Chunk JSON files saved to disk (source for BM25, Graph Builder, RAGAS evaluation)
- Neo4j vector store: :Chunk nodes with HNSW index on 768-dim BAAI/bge-base-en embeddings.

TIER 2 — KNOWLEDGE GRAPH (Neo4j Aura)

- Hand-crafted medical schema: Cancer, ChemoDrug, EatingAdverseEffect, FoodItem, NutritionGuideline, NonChemoDrug, DrugInteraction, TreatmentProtocol, Ailment
- Six pre-written Cypher query templates for six intent categories: cancer drugs+effects, food guidance, non-chemo interactions, protocol details, drug–drug interactions, fulltext general
- Deterministic intent detection: exact substring matching against KNOWN_CANCERS, KNOWN_CHEMO_DRUGS, KNOWN_NON_CHEMO_DRUGS, FOOD_KEYWORDS, INTERACTION_KEYWORDS
- Fulltext index (chemo_text_index) as fallback when primary Cypher returns empty results
- Graph context formatted as [G1], [G2] blocks; placed before vector chunks in LLM prompt

TIER 3 — RETRIEVAL ENGINE

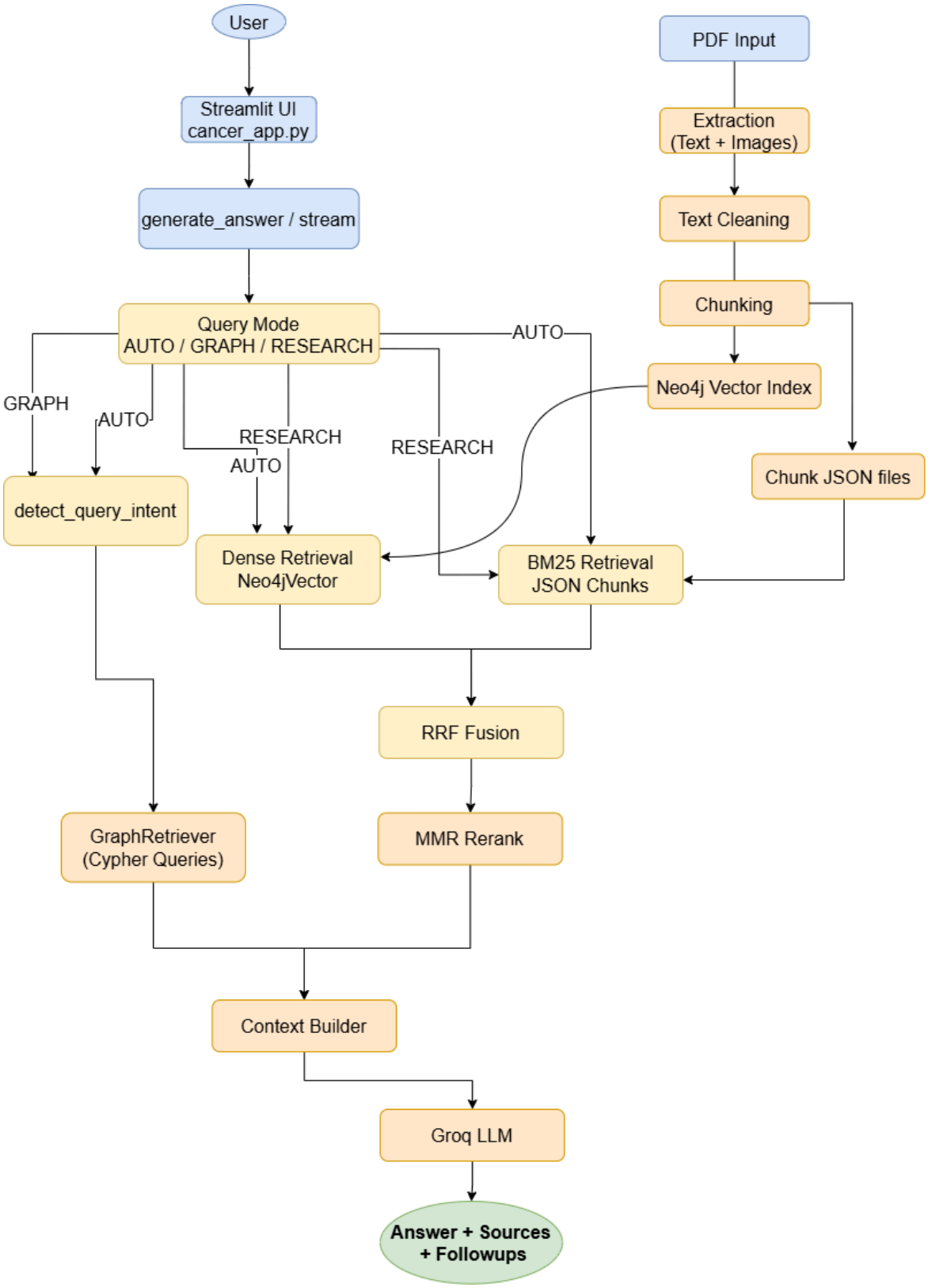
- Dense retrieval: Neo4jVector.as_retriever(k=20) — semantic search via HNSW
- Sparse retrieval: BM25Retriever(k=20) — lexical search over chunk JSON files on disk
- RRF fusion: Reciprocal Rank Fusion merges dense + sparse → top 20 candidates; scores normalised
- MMR reranking: Maximal Marginal Relevance (λ=0.6) → 8 final content chunks; cosine similarity computed locally
- Image channel (parallel): Separate BM25 index of has_image_tags=True chunks → top 2 image chunks → injected as "## Visual References" section; BYPASSES MMR entirely
- Context assembly: [Graph context] + [Content chunks 1..8] + [Visual References IMG1..IMG2]

TIER 4 — GENERATION & DELIVERY

- Out-of-corpus detection: 15 regex patterns (FDA approval, vaccine, 202x dates, recent trials) → proactive web search BEFORE retrieval
- LLM prompt: mode-specific instructions + mandatory [IMAGE:] tag directive + context-only constraint + disclaimer requirement
- Groq LLaMA-3.3-70b-versatile: answer generation (streaming for UI; blocking for evaluation)
- Web fallback (DuckDuckGo): clean presentation — web answer stands alone if RAG empty; labelled as "Additional sources" if RAG partial

Previous Architecture :

Current Architecture:

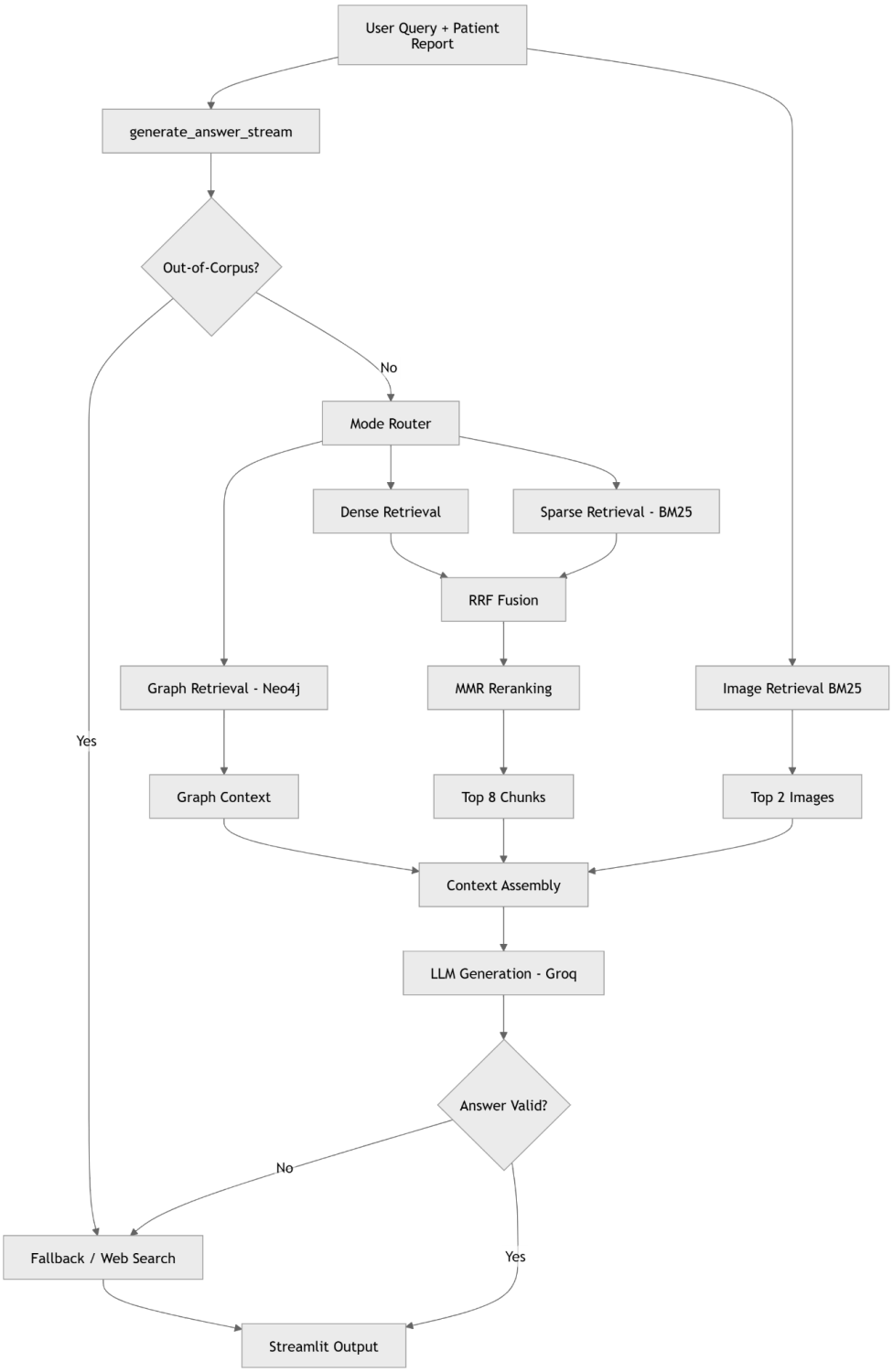


5.2 Data Flow Diagram

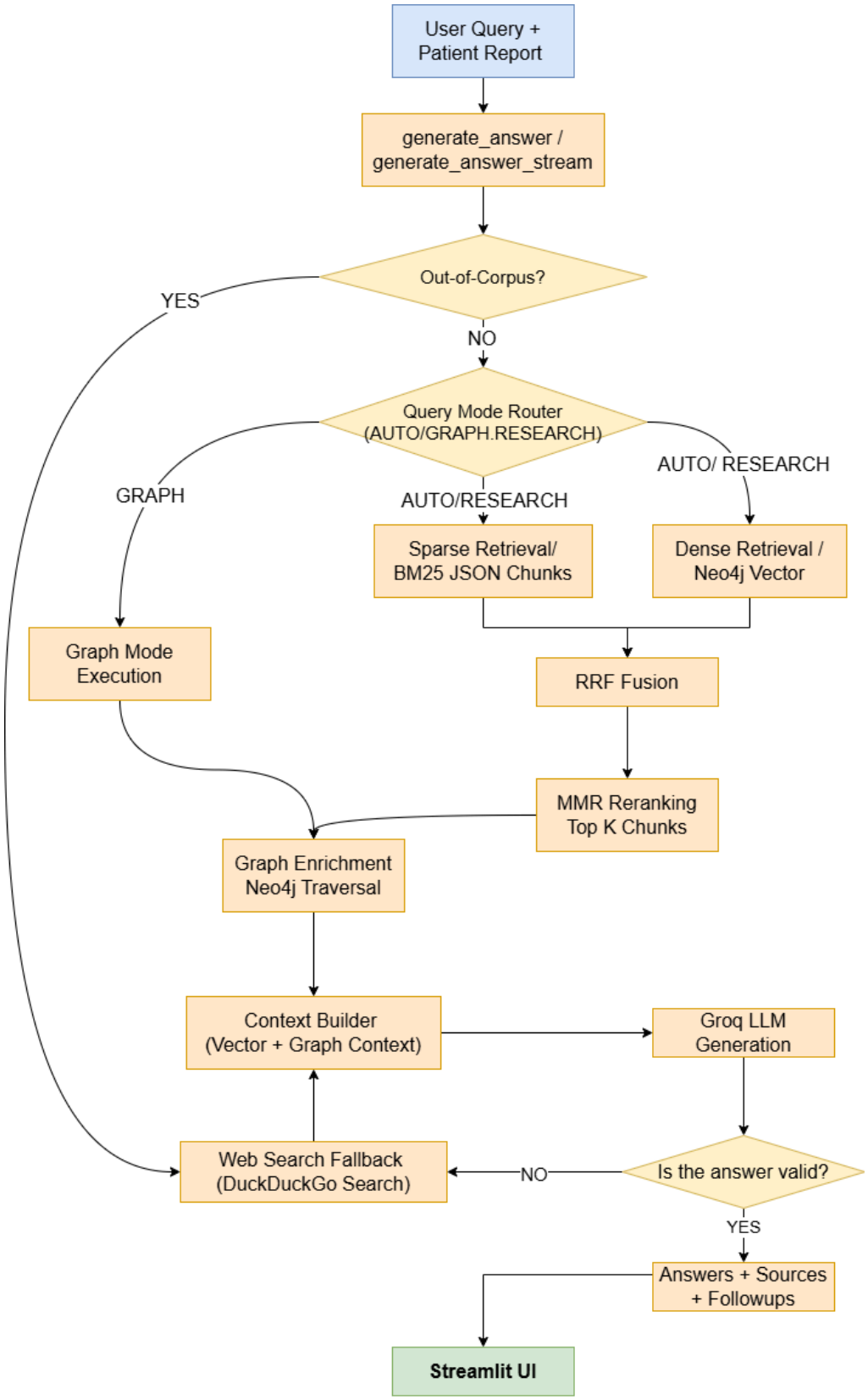
The following table traces the exact path of a query through the system, showing which components are invoked at each step:

Step	Component Invoked	Data In / Out
1. Query received	cancer_app.py → generate_answer_stream()	User query string, patient_report, cancer_filter, query_mode
2. Out-of-corpus check	_is_out_of_corpus_query()	15 regex patterns matched → if True: skip retrieval, go to step 10
3. Mode routing	if/elif on query_mode constant	Routes to _run_research_mode / _run_graph_mode / _run_auto_mode
4a. Graph retrieval (GRAPH/AUTO)	GraphRetriever.retrieve() → Cypher templates	intent_info dict → Cypher params → Neo4j session.run() → rows → _format_graph_context()
4b. Dense retrieval	Neo4jVector.as_retriever(k=20)	Embedded query vector → HNSW ANN search → 20 Document objects
4c. Sparse retrieval	BM25Retriever(k=20)	Tokenised query → BM25 scoring → 20 Document objects
5. RRF fusion	reciprocal_rank_fusion(dense, sparse)	40 docs → RRF scores → top 20 merged candidates
6. MMR rerank	mmr_rerank(query, candidates, embed_model)	20 candidates → cosine similarity matrix → 8 final content chunks
7. Image retrieval (parallel)	_retrieve_image_chunks(query)	Image-only BM25 index → top 2 image chunks → bypass MMR
8. Context assembly	build_context(vector_docs, graph_ctx, image_docs)	Graph [G1..Gn] + Content [1..8] + Visual References [IMG1..2]
9. LLM generation	Groq API → streaming	Prompt (mode instruction + context + question) → streamed tokens → full_answer
10. Fallback check	_rag_has_no_answer() / _is_out_of_corpus_query()	If triggers: _web_search_fallback() → DuckDuckGo → second LLM call
11. Response delivery	generate_answer_stream() → Streamlit write_stream()	Tokens yielded to UI; sources/followups stored in session_state

Previous DFD:



Current DFD:


User Query + Patient Report
generate_answer / generate_answer_stream
Out-of-Corpus?
YES
NO
Query Mode Router (AUTO/GRAPH.RESEARCH)
GRAPH
AUTO/RESEARCH
AUTO/ RESEARCH
Sparse Retrieval/ BM25 JSON Chunks
Dense Retrieval / Neo4j Vector
Graph Mode Execution
RRF Fusion
MMR Reranking Top K Chunks
Graph Enrichment Neo4j Traversal
Context Builder (Vector + Graph Context)
Groq LLM Generation
Web Search Fallback (DuckDuckGo Search)
NO
Is the answer valid?
YES
Answers + Sources + Followups
Streamlit UI

5.3 Knowledge Graph Schema

The Neo4j knowledge graph encodes the clinical nutrition and drug interaction domain for oncology patients. This schema was hand-crafted based on clinical domain research — it was not auto-generated by an LLM.

Node Label	Key Properties	Relationships (→ target)
Cancer	name, subtype	→ TREATED_WITH → ChemoDrug
ChemoDrug	name, drug_class, notes, mechanism	→ CAUSES_EATING_EFFECT → EatingAdverseEffect
EatingAdverseEffect	name, management_tip	→ WORSENED_BY → FoodItem; → RELIEVED_BY → FoodItem
FoodItem	name, category	Target of WORSENED_BY / RELIEVED_BY
NutritionGuideline	text (e.g. "Folic acid 400-1000mcg daily from 7 days before pemetrexed")	→ REQUIRED_FOR → ChemoDrug; → MANAGES → EatingAdverseEffect
NonChemoDrug	name, drug_class	→ HAS_INTERACTION_WITH → ChemoDrug; → DESCRIBED_BY → DrugInteraction; → TREATS → Ailment
DrugInteraction	severity, description, clinical_action, eating_relevance, mitigation	→ COMPOUNDS_EATING_EFFECT → EatingAdverseEffect
TreatmentProtocol	name, cancer, description, setting	→ INCLUDES_DRUG → ChemoDrug
Ailment	name	Target of TREATS

5.4 Intent Detection Logic (Six-Query Cypher Library)

Rather than using an LLM to generate Cypher on the fly (error-prone with open-source models), six pre-written templates handle all clinical nutrition query patterns. The intent router is a simple, deterministic priority chain:

Priority	Intent & Cypher Template
1 (highest)	NonChemoDrug + interaction signal → INTENT_NON_CHEMO_INTERACTION: traverses NonChemoDrug → DrugInteraction → EatingAdverseEffect
2	Protocol name detected → INTENT_PROTOCOL_DETAIL: retrieves TreatmentProtocol with all INCLUDES_DRUG drugs, eating effects, and mandatory guidelines
3	ChemoDrug + food signal → INTENT_FOOD_GUIDANCE: full food/avoid/eat/guideline lookup per drug
4	Cancer name + food/drug signal → INTENT_CANCER_DRUGS_EFFECTS: cancer → all treatment drugs → all eating effects
5	ChemoDrug alone → INTENT_FOOD_GUIDANCE (same template as priority 3)
6	Cancer name alone → INTENT_CANCER_DRUGS_EFFECTS
7 (fallback)	Eating effect keyword or no match → INTENT_GENERAL_GRAPH: fulltext index search on chemo_text_index

6. Key Features

Feature	Description	Technical Mechanism	Clinical Value
Three-Mode Retrieval	RESEARCH / GRAPH / AUTO mode selector in sidebar	Explicit routing to PDF-only, graph-first, or fused paths	Clinician picks Research; patient picks Graph; Auto for mixed needs
Graph-Grounded Drug–Food Guidance	Structured interaction and nutrition data from Neo4j	Six Cypher templates with typed entity traversal	Answers "what to eat/avoid on cisplatin" with verifiable graph sources
Image & Figure Retrieval	[IMAGE: filename.png] tags rendered as actual images in UI	Dedicated image BM25 index bypasses MMR; ## Visual References section in prompt	PRISMA flowcharts, Kaplan-Meier curves, treatment decision trees displayed inline
Patient Report Analysis	Upload PDF/text → structured 6-part clinical summary	PyMuPDF extraction + AUTO mode with 6-section prompt template	Diagnosis, staging, treatment plan, nutrition guidance from one upload
Proactive Web Fallback	Out-of-corpus queries (FDA approvals, recent trials) go to web	15 regex patterns trigger DuckDuckGo before retrieval	Prevents hallucination on time-sensitive regulatory information
Three-Tier Fallback Detection	Detects "I don't know" even when buried in long answers	Short-answer threshold + multi-phrase count + 5 regex patterns	Catches partial hallucinations that phrase-matching alone misses
Streaming UI	Answer tokens streamed live with image rendering	Groq streaming API + Streamlit write_stream() + post-stream [IMAGE:] re-render	Responsive UI even for long clinical explanations
RAGAS Evaluation Suite	20-question golden dataset with 4 categories, HTML report	faithfulness, answer_relevancy, context_precision, context_recall + custom image_score, web_score	Quantifies pipeline quality; per-run history for regression detection

7. Tools & Technologies

7.1 Core Stack

Component	Technology	Version/Notes	Role	Alternative Evaluated
Embeddings	BAAI/bge-base-en	110M params, 768-dim	Text embedding for dense retrieval and MMR	nomic-embed-text (Ollama) — slower on CPU
Vector DB	Neo4j Aura (Free)	HNSW vector index on :Chunk nodes	Combined vector store + knowledge graph	Qdrant (v3), FAISS, Milvus
LLM (Generation)	Groq LLaMA-3.3-70b-versatile	Free API tier, 128k context	Answer generation (streaming + blocking)	LLaMA-3.1-8b-instant (faster, weaker reasoning)
LLM (Evaluation)	Groq LLaMA-3.1-8b-instant	RAGAS judge; faster	RAGAS faithfulness + relevancy scoring	LLaMA-3.3-70b (more accurate but slower)
PDF Extraction	Docling + PyMuPDF + RapidOCR	Docling for tables, PyMuPDF for images, RapidOCR for scanned pages	Primary ingestion pipeline	PyPDF, PDFPlumber, PyMuPDF4LLM (all inferior for complex docs)
Lexical Retrieval	BM25Retriever (LangChain)	From chunk JSON files on disk	Sparse retrieval for exact matches	TF-IDF (weaker)
LLM Framework	LangChain + LangChain-Neo4j	langchain-community, langchain-groq, langchain-huggingface	Retriever interfaces, embeddings wrappers, prompt management	LlamaIndex
UI	Streamlit	Streaming mode, session_state	Chat interface with image rendering and file upload	FastAPI + React
Evaluation	RAGAS v0.2.6 (explodinggradients)	! Must be explodinggradients fork, NOT vibrantlabsai fork	Faithfulness, relevancy, precision, recall metrics	TruLens (planned for production monitoring)
Web Fallback	DuckDuckGo (duckduckgo-search)	No API key required	Web search for out-of-corpus queries	SerpAPI, Tavily (more reliable; future upgrade)
Image Captioning (Early)	Ollama llava:7b / Moondream2	Local VLM inference	Generate image descriptions for embedding	Replaced by [IMAGE:] tag injection approach

Setup Guides.

Companion quick-start documents have been prepared for the two primary cloud services in this stack.

The **Neo4j AuraDB Quick Start Guide (click here)** covers account registration, free-tier instance provisioning, credential management, Cypher query basics, and the instance-pausing behaviour that affects Graph RAG pipelines.

The **Groq API Quick Start Guide (click here)** covers account setup, API key creation, free-tier rate limits, and the Python integration pattern used throughout this project. Both guides are beginner-friendly and suitable as onboarding references for new contributors.

7.2 Hardware Context

All development was conducted on a severely constrained hardware profile, which shaped many architectural decisions:

Hardware Component	Specification & Impact
Processor	Intel i5 11th Gen - no AVX-512; limits batch sizes for embedding model; ~200ms per embed_documents call
GPU	None - GPU-accelerated models (FP32 transformers, CUDA-dependent libs) entirely unusable; all inference CPU-bound
RAM	<10 GB available after OS overhead — forced sequential processing, explicit gc.collect(), lazy loading pattern
Storage	Windows filesystem with VS Code - higher background RAM pressure than Linux alternatives
Consequence	Architecture optimised for cloud-API generation (Groq) + lightweight local embedding (bge-base-en) + disk-based BM25 - zero local GPU requirements

8. Methodology

8.1 Ingestion Pipeline

The ingestion pipeline transforms raw clinical PDFs into a dual representation: a vector index in Neo4j Aura (for semantic retrieval) and structured JSON chunk files on disk (for BM25 and graph building). It runs once per corpus update and takes approximately 2–5 minutes per paper on the target hardware.

Stage 1: Text Extraction

Docling is the primary extractor. If it returns <300 characters of clean text, PyMuPDF takes over as fallback. The page classifier (PageProfile) categorises each page as text-rich, scanned, mixed, or vector-heavy, routing to the appropriate strategy:

Page Type	Extraction Strategy
text-rich (≥150 chars)	page.get_text() — direct PyMuPDF text extraction
scanned (no text, no images)	EasyOCR with DPI=300; lazy-loaded, GPU=False; language=English
mixed (text + images/vectors)	get_text() + image/vector extraction in parallel
vector-heavy (>25 drawings)	page.get_pixmap(dpi=150) → rasterise entire page; dedup + noise filter

Stage 2: 12-Step Text Cleaning

Each step addresses a specific corruption pattern observed in real medical PDFs:

Step	Problem Addressed
1. Mathematical bold digit repair	Unicode U+1D7CE–U+1D7D7 block renders as garbage in PDF extraction; maps each to ASCII digit
2. NFKC Unicode normalisation	Normalises composed characters, full-width forms, compatibility characters
3. Raw glyph code removal	/uniFB01 → "fi", /uniFB02 → "fl", /C15 → "-"; common in Postscript-based PDFs
4. Unicode ligature replacement	fi→"fi", fl→"fl", ff→"ff", ffi→"ffi", ffl→"ffl", ft→"st"
5. Broken word stitching	Re-joins words split at ligature boundaries: "cisplatin" → "cisplatin"

6. Docling placeholder removal	Removes empty image placeholder tokens inserted by Docling for visual items
7. Running header/footer removal	Regex for "guidelines version", "© YYYY", "Page N of M" patterns on short lines
8. Author running headers	"Smith et al. XXXX" patterns that repeat on every page
9. DOI/URL line removal	Lines starting with "https://", "doi:", "www." — reference noise
10. References section cutoff	Detects "References", "Bibliography", "Works Cited" heading → truncates text there
11. Excess whitespace normalisation	Collapses 2+ spaces to one; collapses 3+ newlines to two
12. Isolated page numbers	Lines containing only 1–3 digits → removed

Stage 3: Image Extraction & Noise Filtering

The image extraction pipeline evolved significantly over 13 weeks. The final approach avoids local VLM inference entirely in favour of caption-based [IMAGE:] tag injection.

Images are extracted via Docling's PictureItem/TableItem elements (with surrounding context text for caption detection) or via PyMuPDF's get_images() for fallback. Every extracted image passes through a seven-layer noise filter before being saved:

Filter Layer	Rejection Criteria
Layer 1: Size & Aspect	Width or height <100px; aspect ratio >6.0; small square badge ≤200px
Layer 2: Colour Analysis	BW ratio >70% with high edge density (QR codes); Green ratio >45% (ORCID badges); Teal >35% (check-mark icons); Orange >30% on small images (journal badges); Sepia >35% on small images (publisher logos); BW ratio >55% on small images (logo plates)
Layer 3: Flat Colour	Edge ratio <5% on images <300px — solid colour icons/dividers
Layer 4: pHash Blocklist	Perceptual hash within Hamming distance 10 of any previously rejected image
Layer 5: Context Text	Surrounding text matches publisher/journal patterns (Elsevier, MDPI, Springer, © 2024, ISSN, etc.)
Layer 6: Portrait Cover	Large portrait-format image (>400px) with strong dominant hue — journal cover detection
Layer 7: Override	If context text matches "Figure N", "Table N", "Kaplan-Meier", "survival", etc. — always keep regardless of other layers

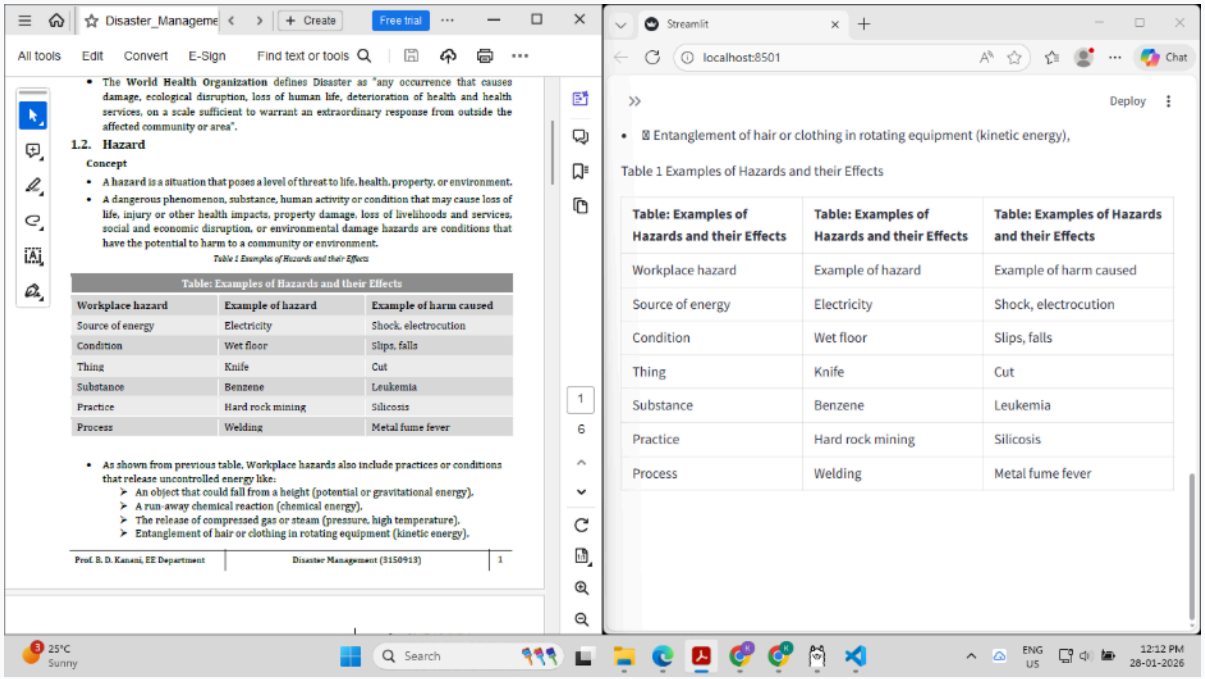


Figure 1: Docling + Ollama output on IBM SAM ESSO Practices document

This was the initial attempt at image description generation. Docling successfully structured the text and tables, but the Ollama llava:7b integration failed to produce image descriptions — only detected image placement. This failure led to the pivot towards the [IMAGE:] tag injection approach.

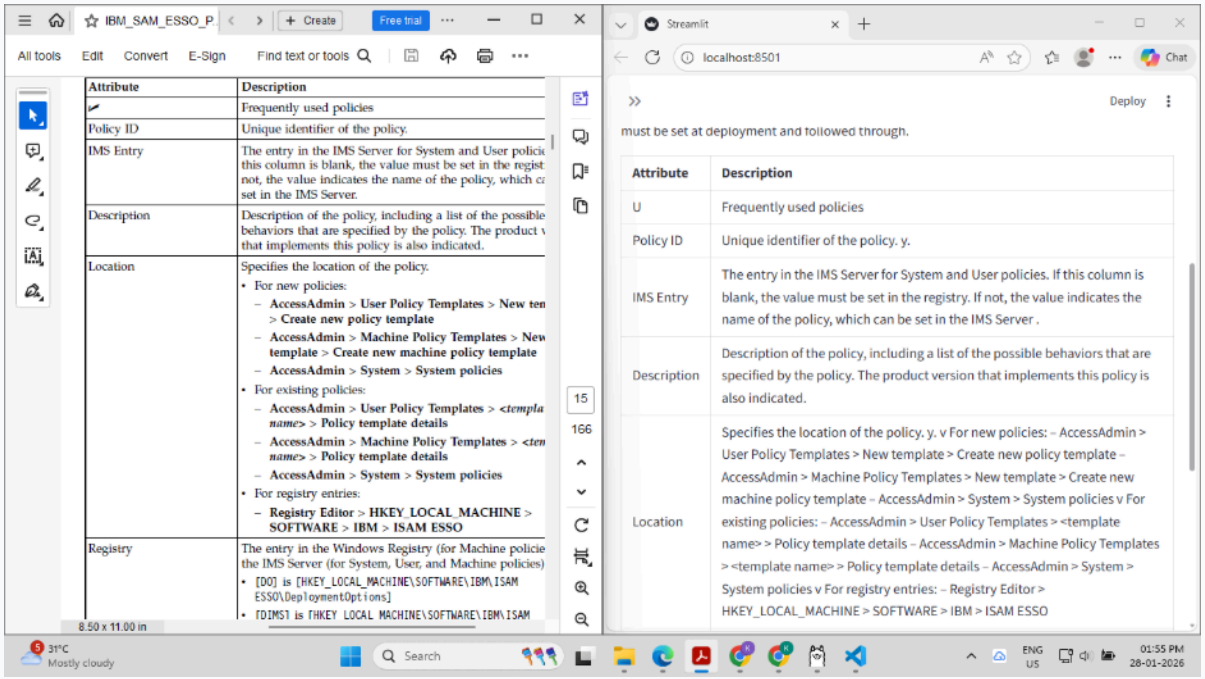


Figure 2: Docling + Ollama on IBMMaaS360_Best_Practices_for_Policies.pdf

Second test document chosen specifically for its complex mix of tables and embedded images. Table extraction was highly accurate with Docling. Image description still not working — triggering the search for a combined pipeline.

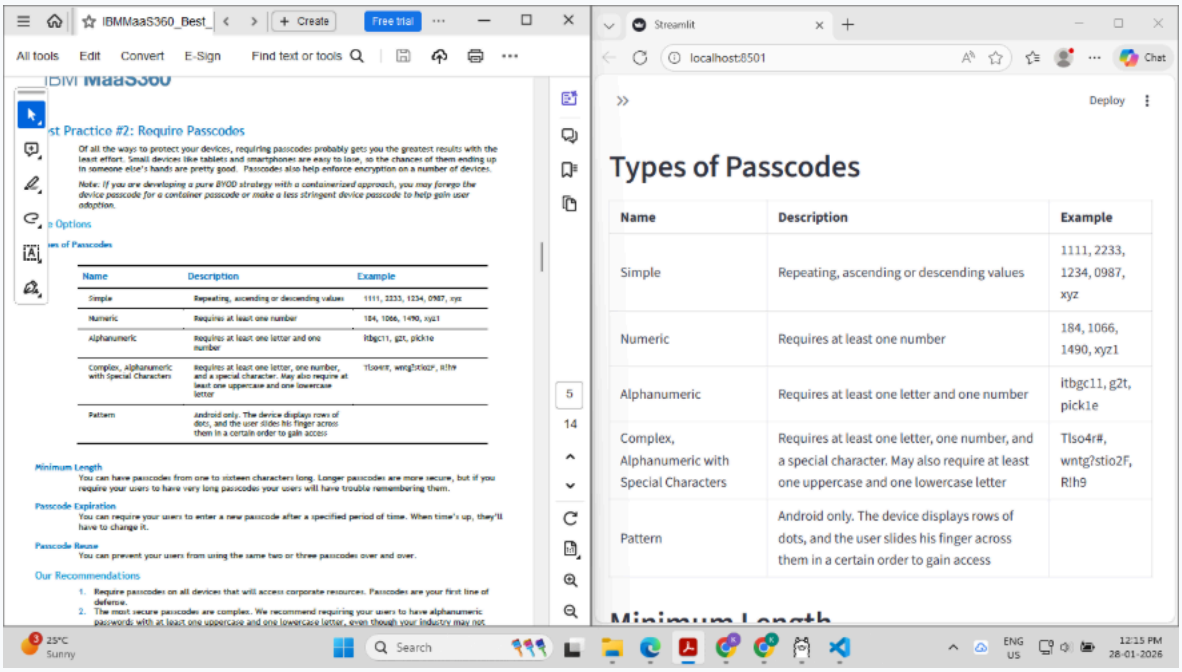


Figure 3: Further Docling + Ollama output from IBMMaaS360 document

Additional pages from the same test. Confirms Docling table quality; highlights the image description gap.

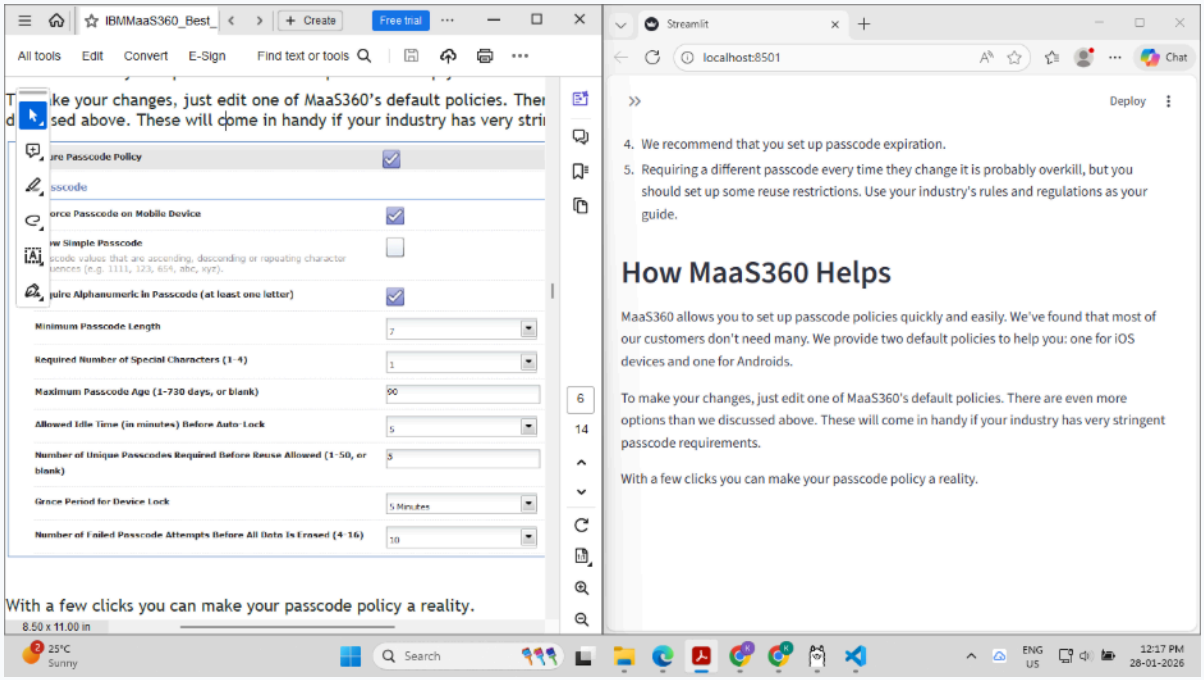


Figure 4: First attempt at image extraction — unsuccessful

Trial 1: PyMuPDF + Ollama llava:7b combination. Images extracted; descriptions generated. Failure: 3–4 minutes per page processing time — impractical for a 200-page clinical review paper.

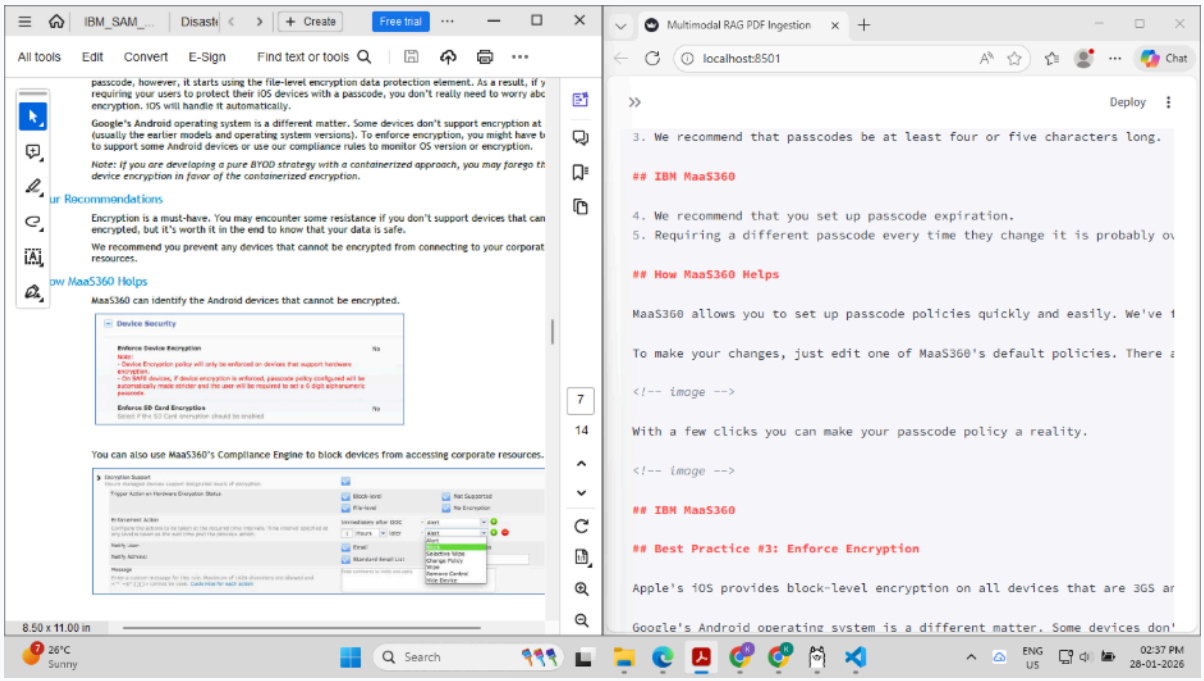


Figure 5: Image placement detected; description not yet available

Trial 2: Claude-assisted image detection (qwen3-vl:7b). Successfully detected image locations in document structure. However qwen3-vl:7b could only detect locations, not describe content. Model was replaced with llava:7b for the next trial.

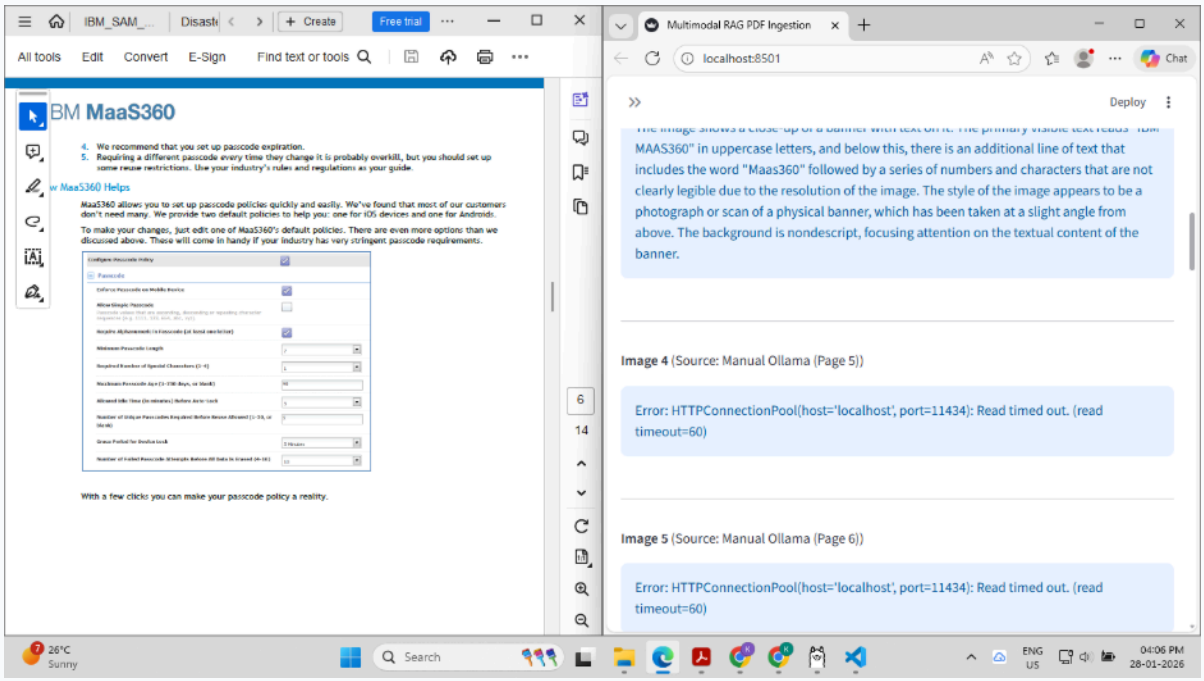


Figure 6: Manual VLM description attempt — timed out

Attempting a manual VLM pipeline with full-resolution images. Execution time exceeded 10–15 minutes without producing output on the target i5 hardware. This conclusively ruled out local VLM inference at any production scale.

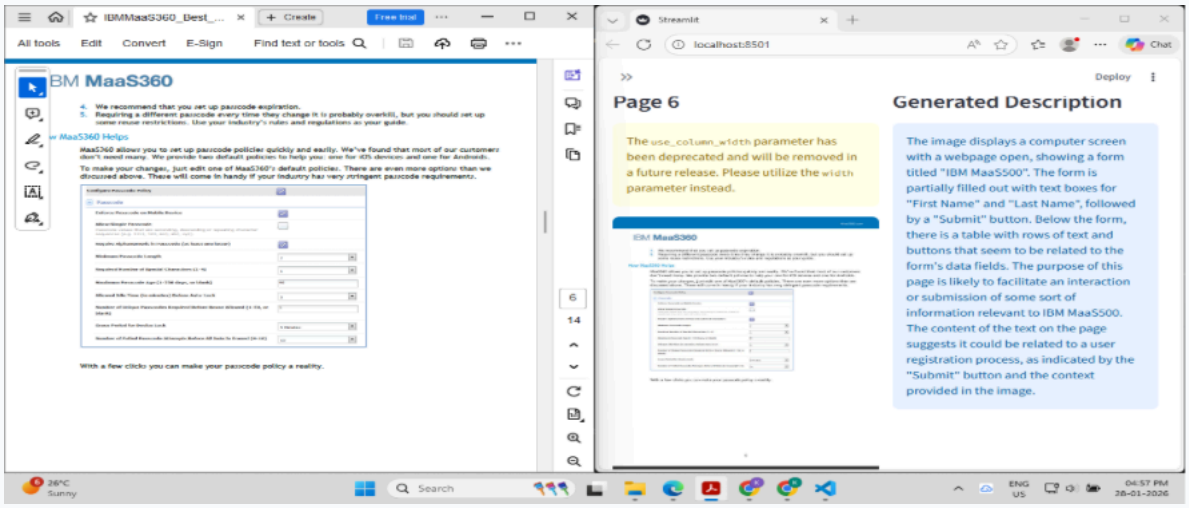


Figure 7: PyMuPDF + llava:7b — accurate output but slow

Final VLM trial: accurate image descriptions generated! However 2–3 minutes per page remained impractical. This result directly motivated the architectural decision to replace VLM-based captioning with the [IMAGE:] tag injection approach for the production pipeline.

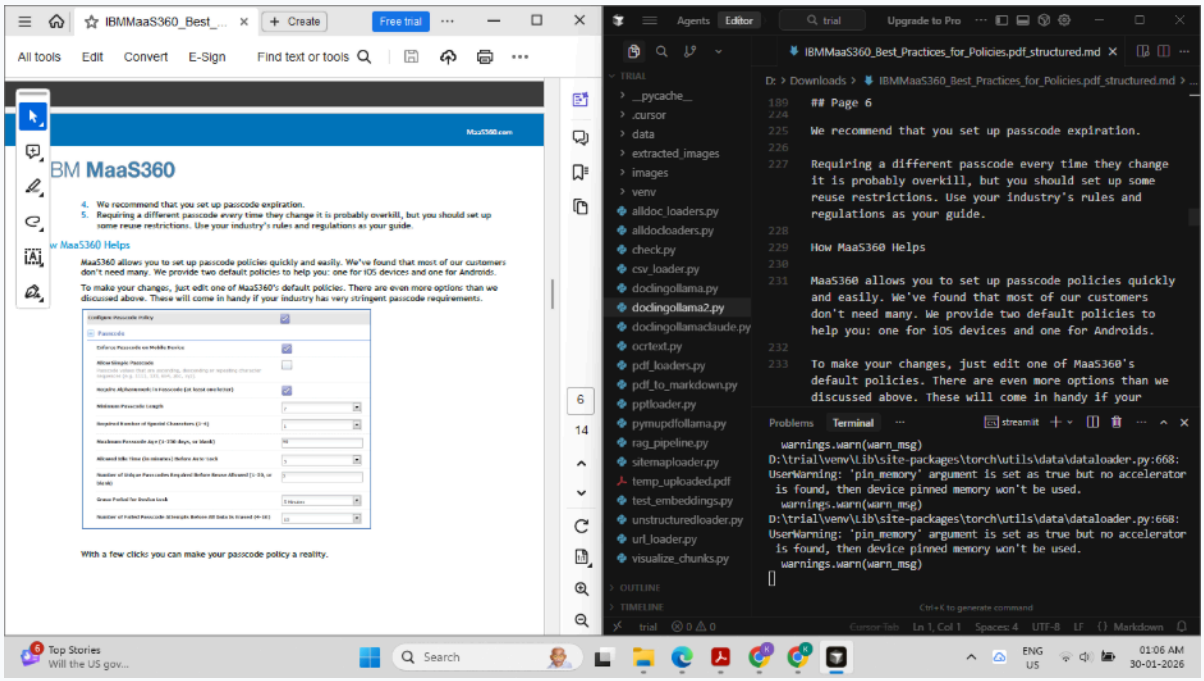


Figure 8: Successful output — Docling + PyMuPDF + RapidOCR pipeline

Final ingestion pipeline producing both accurate table extraction (Docling) and image extraction (PyMuPDF) with OCR for scanned pages (RapidOCR). Total initialisation ~3 minutes; subsequent pages fast.

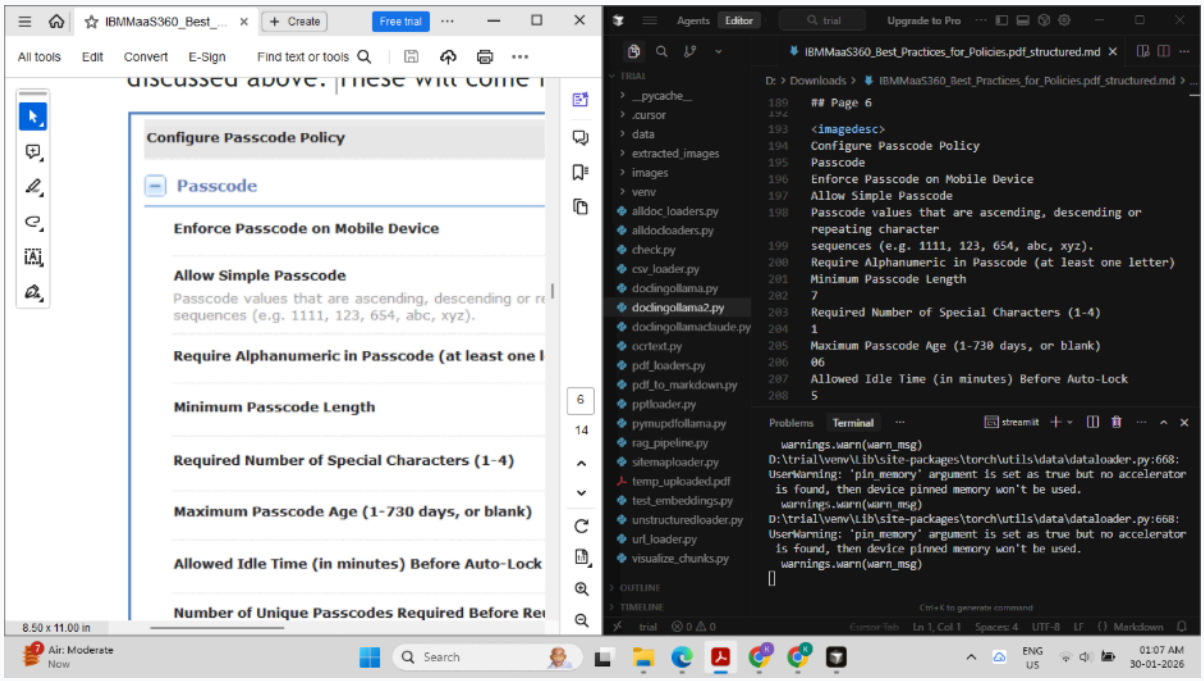


Figure 9: Further output from the finalised Docling + PyMuPDF + RapidOCR pipeline

Additional pages demonstrating consistent extraction quality across different PDF layouts.

Key Finding: The [IMAGE:] tag approach — appending a "## Extracted Visual Assets Database" section with tag-and-caption entries to the markdown before chunking — is the breakthrough that made image retrieval work without local VLM inference. BM25 finds tags by text; the LLM sees them as context; Streamlit renders them as images. No separate image database required.

Stage 4: Two-Stage Chunking

The chunking strategy was selected after evaluating five approaches against the requirement to preserve section context in metadata while producing consistent chunk sizes for the clinical domain:

Strategy	Context Preservation	Chunk Consistency	Decision
Fixed-size (character)	Low - arbitrary splits mid-sentence	High (by design)	Rejected: severs clinical sentences
Recursive character splitting	Medium-High: respects sentence/paragraph boundaries	High: consistent sizes	Used as Stage 2
Document structure (header-aware)	High: preserves section boundaries	Variable: depends on section length	Used as Stage 1
Semantic (embedding-based)	Very High: semantic topic boundaries	Low: variable sizes	Not used: too slow on CPU; no significant quality gain for structured medical PDFs
Agentic splitting	Highest: AI determines boundaries	Unpredictable	Future scope only

The final implementation uses MarkdownHeaderTextSplitter (H1/H2/H3 boundaries) followed by RecursiveCharacterTextSplitter (chunk_size=1200, chunk_overlap=200). This produces chunks with accurate section_hierarchy metadata - essential for the LLM to know whether it is reading from Methods, Results, or Discussion.

Figure 10: JSON representation of extracted chunks

Output showing chunk_id, source_file, cancer_type, content_type, section_hierarchy, char_length, has_image_tags fields. The has_image_tags boolean is what populates the dedicated image BM25 index.

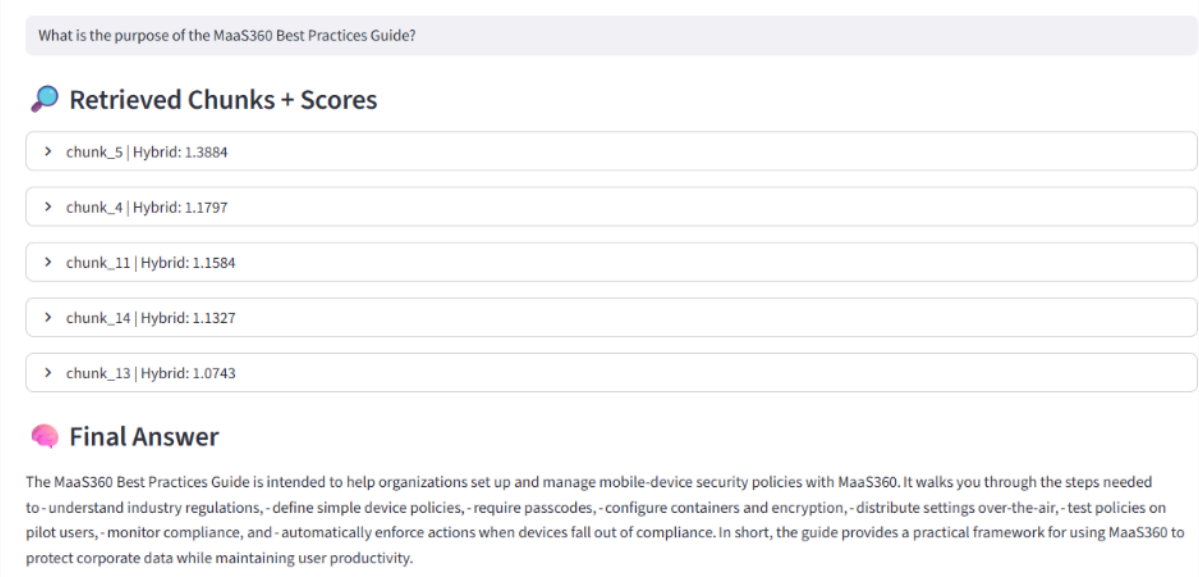


Figure 11: Chunk storage structure in output directory

Shows the output/chunks/ directory organisation with per-paper _chunks.json files and output/captions/ with _caption_chunks.json files.

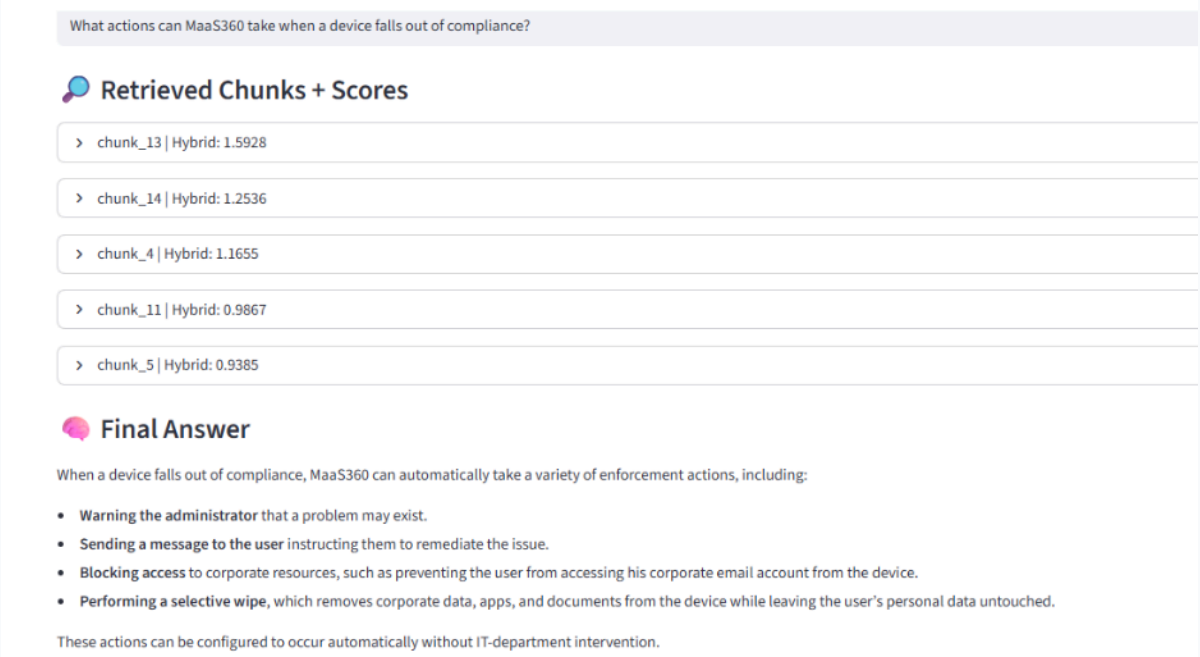


Figure 12: Chunking output — initial trial

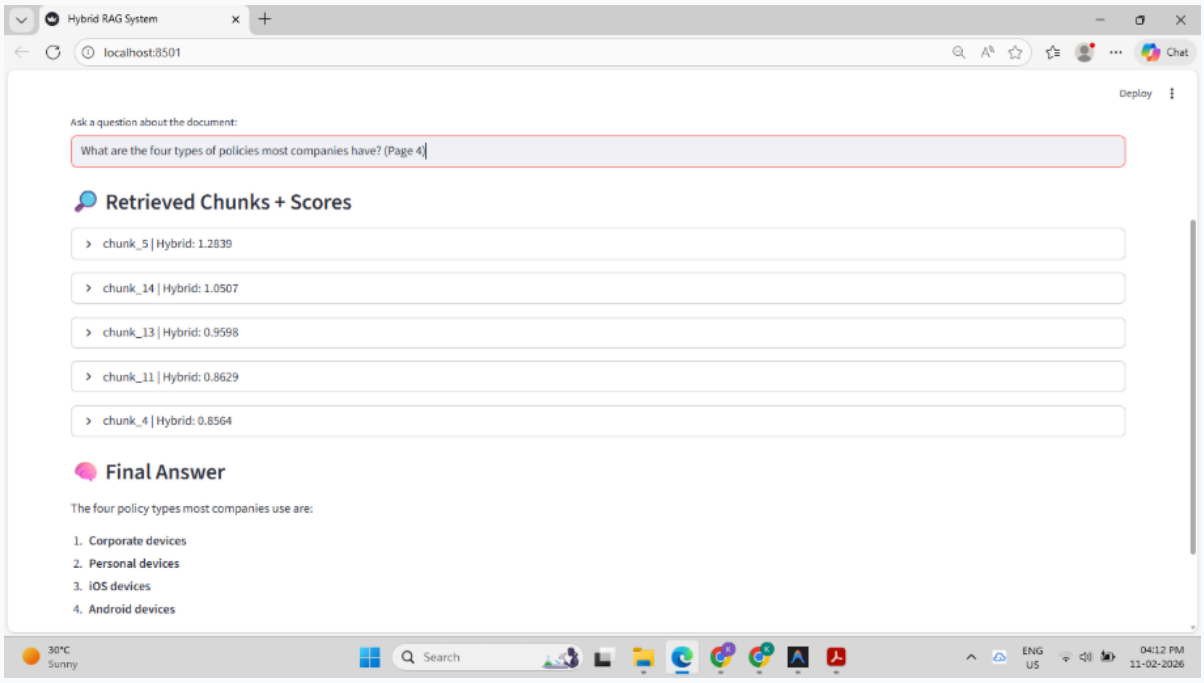


Figure 13: Chunking output — further analysis showing section hierarchy metadata

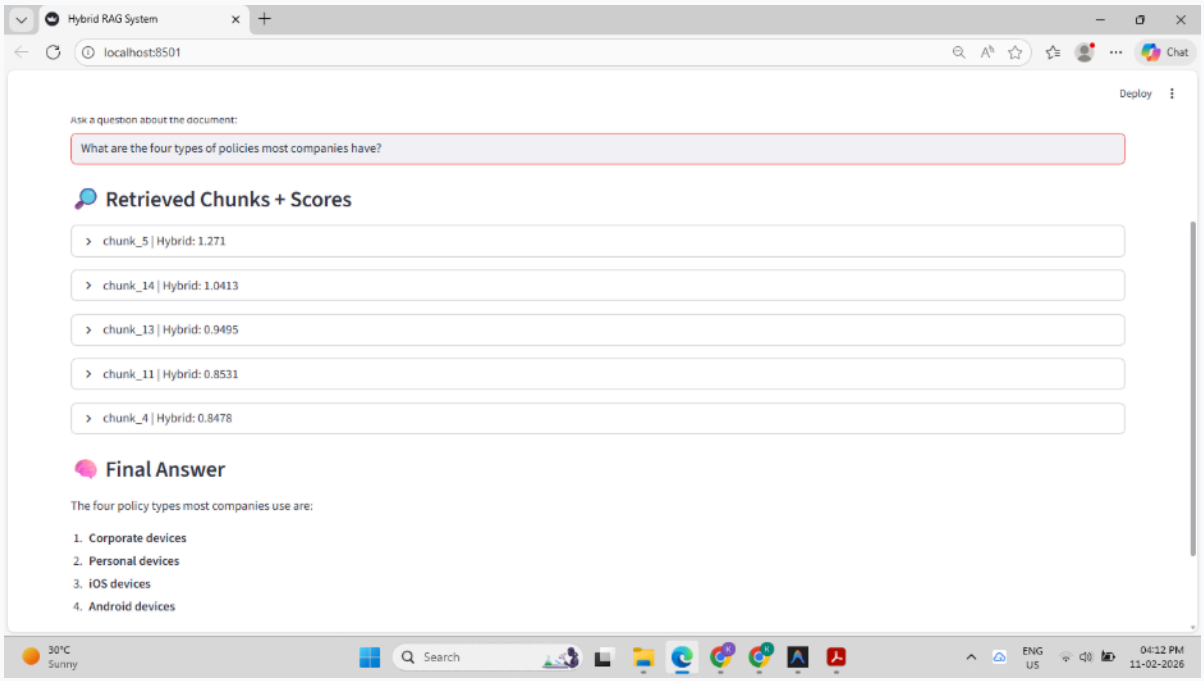


Figure 14: Additional chunking result from complex multi-column PDF

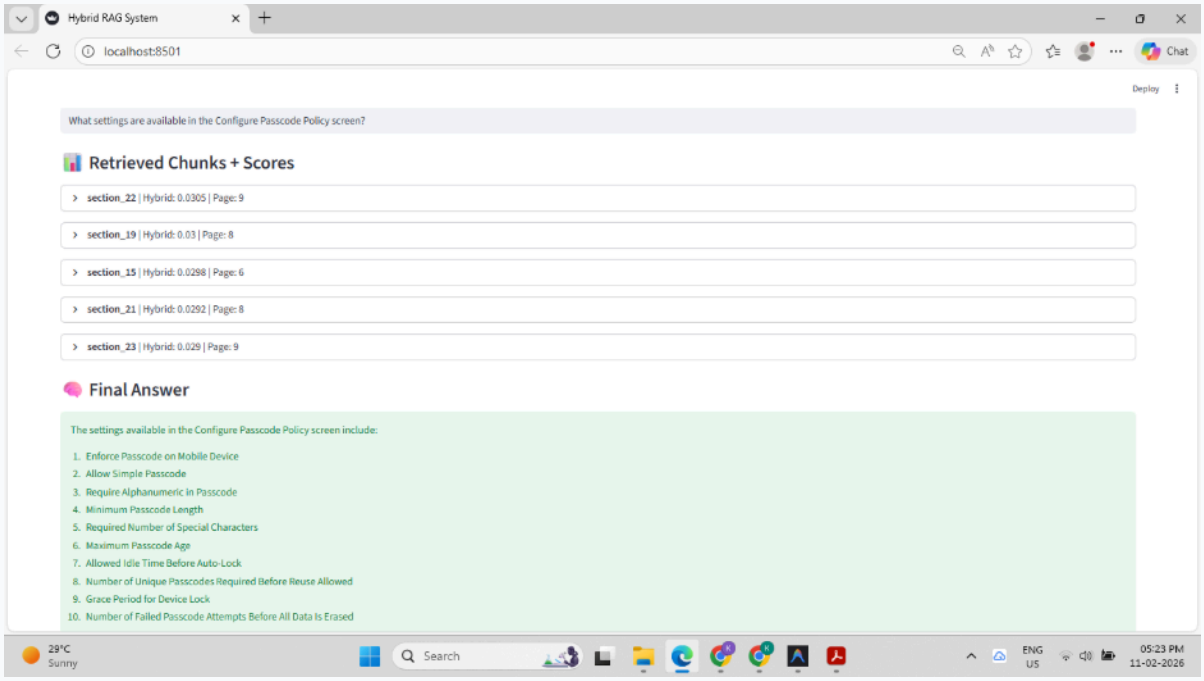


Figure 15: Trial 2 — Successful output with images and tables handled simultaneously

The breakthrough output: text chunks with section hierarchy, caption chunks with image_path, and [IMAGE:] tags preserved in has_image_tags=True chunks.

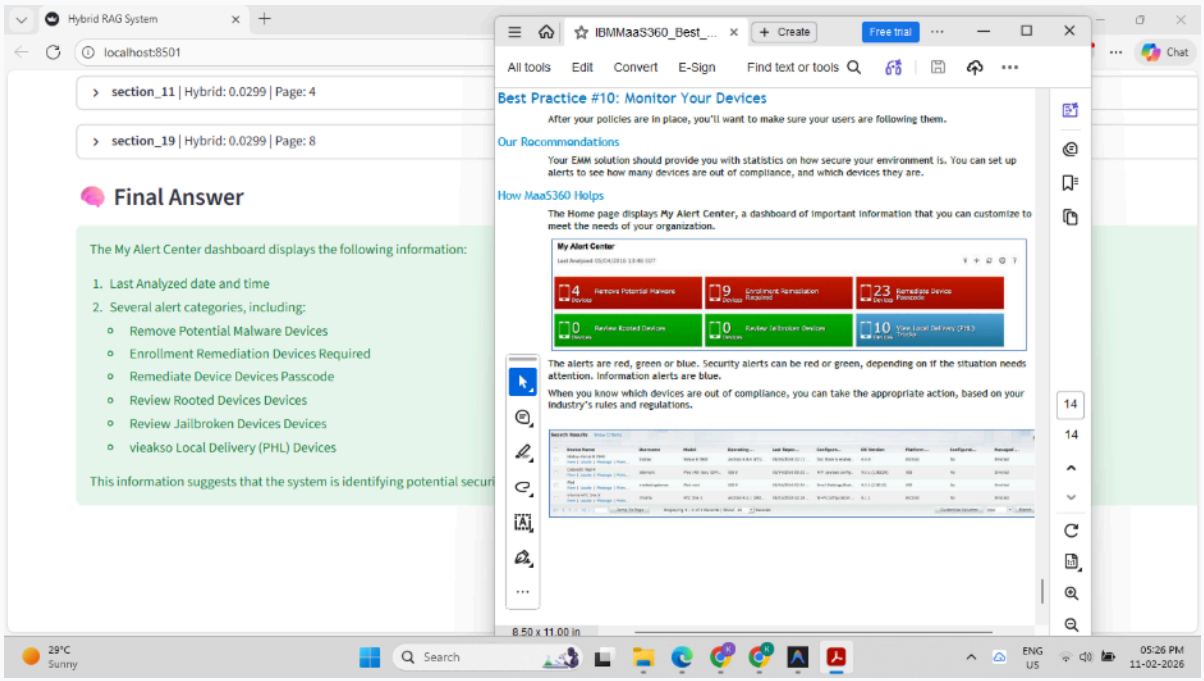


Figure 16: Further successful output from the finalised chunking pipeline

Consistent results across multiple papers confirming pipeline reliability.

8.2 Retrieval Engine

BM25 — Lexical Foundation

BM25Retriever is initialised from the chunk JSON files on disk (not from Neo4j). This keeps it decoupled from the vector store and allows it to be rebuilt independently. The retriever is cached globally after first build.

A second BM25 retriever instance is built from a filtered subset of chunks where has_image_tags=True. This dedicated image index is the key to reliable image retrieval — image chunks are structurally shorter than content chunks and always lost to longer prose chunks under a shared MMR process.

Dense Retrieval via Neo4jVector

Neo4jVector replaces the original Qdrant vector store (v3 of the system). The migration to Neo4j Aura consolidated both the vector index and the knowledge graph into a single database, eliminating cross-database query logic. The vector store is cached in a global dict after first connection to avoid repeated Neo4j handshakes (which cost ~3–5 seconds on the free Aura tier).

Reciprocal Rank Fusion

RRF addresses the score normalisation problem between BM25 (scores 0–10+) and dense retrieval (cosine similarity 0–1). The formula score += 1/(k + rank + 1) where k=60 ensures that a document ranked 1st by either method receives the same score contribution regardless of the absolute similarity value. The 40 candidates from both retrievers are merged and the top 20 by RRF score advance to MMR.

Maximal Marginal Relevance (λ = 0.6)

MMR is applied as a local reranking step after RRF. The key formula:

> **MMR Formula:** score = λ · Sim(query, doc) − (1−λ) · max(Sim(doc, selected_docs)) where λ=0.6 weights relevance slightly above diversity. At λ=1 this is pure similarity search; at λ=0 this is pure diversity. The 0.6 setting was chosen after testing 0.5 and 0.7 — 0.5 produced overly diverse chunks losing clinical coherence; 0.7 produced near-duplicate chunks about the same drug.

MMR produces 8 final content chunks. Testing showed that beyond 8 chunks, LLM context quality degraded (the model started referencing distant chunks incorrectly) while below 6 chunks recall suffered on multi-hop questions.

8.3 Answer Generation & Web Fallback

Prompt Architecture

The prompt is mode-specific with four structural blocks:

Prompt Block	Content & Purpose
Mode Instruction	RESEARCH: "cite [1],[2] for statistics"; GRAPH: "prioritise [G1],[G2] facts"; AUTO: "use graph for drug specifics, literature for statistics"
Context Block	Graph [G1..Gn] + Content [1..8] + ## Visual References [IMG1..2]; ordering is intentional — relational facts before narrative
Visual References Instruction (mandatory)	"MUST include [IMAGE: filename.png] tags. ALWAYS reference when question asks about figures. ONLY reference filenames explicitly listed in context. NEVER invent filenames."
Answer Instructions	"Answer using ONLY provided context. Cite [G1],[1] etc. If not in context, state clearly. End with oncologist disclaimer."

Three-Tier Web Fallback Detection

Tier	Trigger Condition
Tier 1 (Short answer)	Answer length <300 chars AND any no-answer phrase present → definitely no answer
Tier 2 (Long answer, repeated signals)	Answer length ≥300 chars AND ≥2 no-answer phrases found → LLM admitted ignorance despite long context
Tier 3 (Regex patterns)	5 specific patterns: "i do not have enough information to provide/answer", "cannot provide a specific answer", "not able to confirm/verify", "specific question is not covered/available", "no specific information available"
Proactive (out-of-corpus)	Fires BEFORE retrieval: 15 patterns including \bvaccine\b, \bfda.approv, \b202[3-9]\b, \bphase [123] trial, \bclinical.trial.result, \bnhs.guideline, etc.

Web Fallback Presentation Logic

Scenario	Presentation
RAG empty (rag_is_empty=True)	" Answer sourced from web search (not found in clinical literature database):" → web answer standalone, clean presentation
RAG partial (rag_is_empty=False)	RAG answer shown first → divider → "Additional sources found via web search:" → web answer
DuckDuckGo query construction	Medical terms present in query → send clean; no medical terms → append "cancer oncology" suffix

8.4 Multimodal Processing

Evolution of the Image Strategy

The image strategy went through five distinct iterations over 13 weeks:

Iteration	Approach & Outcome
v1: Ollama llava:7b descriptions	Accurate descriptions but 3–4 min/page — rejected at scale
v2: Moondream2 (faster)	Faster but description quality too low for clinical context; confuses equipment/layout images with clinical figures
v3: PaddleOCR + LLM	Tested but unsatisfactory — OCR for images is not description
v4: TesseractOCR standalone	Unsatisfactory — cannot describe figures, only extract embedded text
v5 (final): [IMAGE:] tag injection	Append "## Extracted Visual Assets Database" with caption+tag to markdown → chunk → BM25-searchable; zero VLM inference; full traceability

The Moondream2 sample output demonstrates the quality gap that made VLM descriptions insufficient for clinical use:

> **Moondream2 Sample Caption (from test):** "The image shows a configuration page that is open on a computer screen. The page has a header at the top with the words CONFIGURE PASSWORD written in white letters. Below this heading are several checkboxes..." — This describes a UI screenshot accurately. However for a clinical Kaplan-Meier survival curve, the same model produced generic descriptions missing key clinical values (p-values, median survival, confidence intervals). For a medical RAG system, this level of clinical specificity is insufficient.

Final Image Retrieval Architecture

The dedicated image BM25 index is built at startup from all chunks where has_image_tags=True OR content contains "[IMAGE:". The retriever always returns top 3 candidates, which are then filtered by query token overlap (≥1 content token from query must appear in chunk content, OR the query must contain explicit visual intent keywords). The final cap is 2 image chunks per query — enough to illustrate answers without bloating context.

8.5 Graph RAG Extension

Why a Knowledge Graph Instead of More Vector Chunks?

The drug–food interaction domain has a fundamentally relational structure. Consider the query: "What foods should a breast cancer patient on AC-T and warfarin avoid?" This requires:

1. Knowing that AC-T includes capecitabine
2. Knowing that capecitabine interacts with warfarin (raises INR)
3. Knowing that capecitabine-induced diarrhoea reduces vitamin K absorption
4. Knowing that reduced vitamin K destabilises warfarin dosing
5. Concluding: avoid sudden changes in leafy greens; maintain consistent vitamin K intake

A vector retrieval over PDF text may find a paragraph mentioning warfarin and capecitabine, but it will not surface the structured causal chain unless exactly that chain appears verbatim in a source paragraph. A graph traversal from NonChemoDrug(warfarin) → HAS_INTERACTION_WITH → ChemoDrug(capecitabine) → CAUSES_EATING_EFFECT → EatingAdverseEffect(diarrhoea) → WORSENED_BY/RELIEVED_BY → FoodItem produces this chain directly.

LLMGraphTransformer vs. Hand-Crafted Schema

Approach	Evaluation
LLMGraphTransformer (auto-extraction)	Tested in v4 of the system. Produced inconsistent node types, missed critical nutrition relationships, created false positive edges. Reliability insufficient for medical use.
Hand-crafted schema (final choice)	Took longer to build but produces verifiable, clinically accurate relationships. Every edge type maps to a real clinical concept. Can be audited by a domain expert. Produces consistent [G1], [G2] context blocks.

Text-to-Cypher vs. Pre-Written Templates

Approach	Evaluation
Text-to-Cypher (LLM generates Cypher)	Tested with multiple open-source models. Produces syntactically valid Cypher ~60% of the time. For multi-hop OPTIONAL MATCH chains (required for drug interaction queries) reliability dropped to ~35%. Inconsistent parameter binding. Not safe for a medical system where a bad query returns empty results silently.
Pre-written Cypher templates (final)	Six templates covering 95% of clinical nutrition queries. Always syntactically valid. Predictable output shape. Parameters bound deterministically. Auditable. The fallback to fulltext index ensures no silent empty responses.

9. Use Cases

User	Query Type	MedChat Response Path
Cancer patient on cisplatin	"What foods should I avoid?" (GRAPH mode)	INTENT_FOOD_GUIDANCE Cypher → EatingAdverseEffect nodes → WORSENED_BY FoodItems → structured [G1] answer with avoid/eat lists
Patient on pemetrexed	"Do I need supplements?" (GRAPH mode)	INTENT_FOOD_GUIDANCE → NutritionGuideline nodes → mandatory folic acid 400-1000mcg + vitamin B12 1000mcg IM answer with mandatory flag
Clinician on AC-T + warfarin patient	"What are the dietary risks?" (AUTO mode)	INTENT_NON_CHEMO_INTERACTION → warfarin → HAS_INTERACTION_WITH → capecitabine → DrugInteraction → severity + action + mitigation
Medical student	"What is the 5-year survival for osteosarcoma?" (RESEARCH mode)	PDF chunks from osteosarcoma review → survival statistics with citations [1],[2]
Researcher	"Show me the PRISMA flowchart" (RESEARCH mode)	Image BM25 → [IMAGE: paper_picture_N.png] tag → Streamlit renders actual figure inline
Patient uploading report	Report PDF upload → automatic analysis	PyMuPDF extraction → AUTO mode with 6-section prompt → diagnosis/staging/treatment/nutrition structured summary
Anyone - edge case	"What vaccine prevents osteosarcoma?" (AUTO mode)	Out-of-corpus detection (\bvaccine\b) → proactive DuckDuckGo search → "no approved vaccine" answer with web sources

10. Challenges & Solutions

10.1 Hardware Constraints

Challenge	Root Cause	Resolution & Trade-off
GPU unavailable — cannot run FP32 transformers	i5 11th Gen, no discrete GPU	Groq cloud API for generation (LLaMA-3.3-70b); CPU-only bge-base-en for embeddings. Trade-off: API dependency; Groq rate limits
<10GB RAM — OOM errors during ingestion	Windows background processes + large model load	Lazy loading: one document at a time; explicit gc.collect() between papers; batch_size=100 for Neo4j upserts
2–3 min/page for local VLM (llava:7b)	No GPU acceleration for vision transformer	Replaced local VLM with [IMAGE:] tag injection approach — zero inference cost; full traceability
Local VLM alternatives (qwen3-vl:7b) too limited	Small models can detect images but not describe them clinically	Pivoted to caption-based tagging; captions extracted from document structure (Docling native captions > sliding window > context fallback)

10.2 Package & Integration Issues

Challenge	Failure Mode	Resolution
ragas PyPI fork conflict	vibrantlabsai fork (ragas 0.4.x): Faithfulness does not inherit from Metric; evaluate() raises "All metrics must be initialised metric objects" at runtime with no clear error message	pip uninstall ragas -y && pip install "ragas==0.2.6" (explodinggradients); add fork detection at startup
LangChain API version drift	Neo4jVector and BM25Retriever import paths changed between LangChain minor versions; silent AttributeError	Pin specific versions in requirements.txt; use langchain-neo4j and langchain-community explicitly
Neo4j Aura cold-start latency	First query after inactivity period: 3–5 second Neo4j connection handshake	Global _VECTOR_STORE_CACHE dict caches Neo4jVector instance after first creation; subsequent calls reuse
Groq TPM rate limits during RAGAS	RAGAS evaluation fires multiple LLM calls per question (faithfulness NLI + answer relevancy embedding); exceeds 6,000 TPM at default concurrency	ctx[:3] during evaluation; time.sleep(2) between pipeline calls; RunConfig(max_workers=1, timeout=180) in evaluate()
spaCy en_core_sci_sm unavailable	Medical NER model requires separate install; not available in restricted environment	Fallback to en_core_web_sm for general NER; for v4 graph enricher. Ultimately replaced by deterministic keyword matching in v5.

10.3 Image Retrieval Failure (v5.0 → v5.1 Critical Fix)

This was the hardest-to-diagnose bug in the project. Image questions returned correct text answers but no [IMAGE:] tags. Root cause: MMR was silently discarding image chunks.

Aspect	Detail
Root cause	Image chunks describe figures/tables — they are shorter (~200 chars) and less semantically similar to clinical question embeddings than full prose paragraphs (~1200 chars). In the shared MMR competition, prose always won.
Symptom	RAGAS image_score = 0.0 to 0.5; answer text mentioned figures but had no [IMAGE:] tags; Streamlit showed no inline images
Diagnosis method	Added explicit logging of context_text in _get_contexts(); confirmed image chunk IDs never appeared in final 8 MMR outputs
Fix implemented	_retrieve_image_chunks() builds a separate BM25 index from image-only chunks; retrieves top 2; injects as ## Visual References section in build_context(); completely bypasses shared MMR
Prompt fix (simultaneous)	Weak "if context contains images, include them" language replaced with "MUST include", "ALWAYS reference", "NEVER invent filenames"; dedicated VISUAL REFERENCES INSTRUCTIONS block added
Result	image_score improved from 0.0–0.5 to 1.0 for explicit image questions; image chunks reliably appear in context

10.4 RAGAS Evaluation Challenges

Challenge	Impact	Mitigation
No golden dataset initially	Uninterpretable scores; no baseline for version comparison	Structured 20-question test set with 4 categories; ground truths written per clinical domain; mingled-10 strategy for quick 5-minute eval
Open-source LLM evaluator inconsistency	Same answer scored differently on consecutive faithfulness runs; undermines reproducibility	Use llama-3.3-70b for final benchmarks; log judge model in every run; run each eval 2+ times
Context window truncation	Large contexts cause LLM truncation → incomplete faithfulness NLI; disproportionately affects recall	Context compression before evaluation pass

Hybrid pipeline failure attribution	Low aggregate score doesn't say whether retrieval or generation failed	Separate pre-rerank, post-RRF, post-MMR tracking; custom image_score + web_score metrics alongside RAGAS
Graph query evaluation	Standard RAGAS metrics don't assess graph reasoning quality	Custom graph evaluation layer planned: entity correctness, relation correctness, graph path consistency

11. Performance Considerations

11.1 Latency Profile

Operation	Typical Latency (CPU-only hardware)
BM25 index build (startup, ~900 chunks)	~2–3 seconds; cached after first call
Image BM25 index build (startup)	~0.5 seconds; cached after first call
Neo4jVector first connection	3–5 seconds (Aura cold-start); cached in _VECTOR_STORE_CACHE
Query embedding (bge-base-en)	~200ms per query on i5 CPU
Dense retrieval (Neo4j HNSW, k=20)	~300–500ms including network to Aura
BM25 retrieval (k=20)	~50ms local
RRF fusion (40 docs)	<10ms local computation
MMR reranking (embed 20 docs + matrix)	~800ms on CPU (20 × 200ms per doc embed is reduced by batching)
Image BM25 retrieval (top 3)	~20ms
Graph Cypher query (Neo4j)	~200–400ms per query including network
Groq LLM generation (streaming start)	~1–2 seconds to first token
DuckDuckGo web search	~2–4 seconds per call
Total (GRAPH mode, fast path)	~3–5 seconds to first streaming token
Total (AUTO mode, full path)	~5–8 seconds to first streaming token

11.2 RAGAS Score Targets

Metric	Target & Clinical Interpretation
Faithfulness ≥ 0.80	Every claim grounded in context — critical for patient safety. Below 0.70: review prompt for hallucination-permissive instructions
Answer Relevancy ≥ 0.75	Answer addresses the question. Below 0.60: routing or prompt issue; check query_mode selection

Context Precision ≥ 0.70	Retrieved chunks contribute to answer. Below 0.60: RRF k or MMR λ needs tuning; too much noise in context
Context Recall ≥ 0.70	Ground truth info present in context. Below 0.60: K_DENSE or K_SPARSE too low; consider cross-encoder reranker
Image Retrieval = 1.00	Binary — image questions must produce [IMAGE:] tags. Any value <1.0: check image BM25 index, prompt visual instruction block
Web Fallback = 1.00	Out-of-corpus queries must trigger. Any value <1.0: update _OUT_OF_CORPUS_PATTERNS list in cancer_retrieval.py

11.3 Memory Footprint

Component	RAM Usage
bge-base-en model (loaded)	~440MB (CPU, FP32)
BM25 index (~900 chunks)	~30MB
Image BM25 index (~150 image chunks)	~5MB
Neo4j driver (connection pool)	~50MB
Groq client + LangChain wrappers	~80MB
Streamlit session state (per user)	~10–30MB depending on chat history
Total estimated peak	~650MB — well within 10GB envelope

12. Scalability & Production Readiness

12.1 Database Architecture for Production (Recommended)

The current POC uses a single Neo4j Aura free-tier instance for both the vector index and the knowledge graph. Production deployment would require the following three-tier database architecture:

Database Layer	Technology	Purpose
Knowledge Vector DB	Milvus (enterprise) or Qdrant (high-speed benchmarks)	Stores chunked medical literature; semantic search for general queries
Ephemeral Session Store	ChromaDB or FAISS (in-memory)	Temporary patient report vectors tagged with session_id; deleted on session termination (HIPAA compliance)
Persistent Chat Memory	Redis	Conversation history for cross-questioning; fast session tracking; TTL-based expiry

12.2 Critical Medical & Privacy Considerations

- Metadata filtering: Tag all knowledge base entries by cancer_type (e.g., type: lung_cancer) to prevent irrelevant cross-domain retrieval
- Data isolation: Never co-mingle different patients' report data within the same index. Use namespace partitioning or separate collections in Milvus/Qdrant.
- PII stripping: Apply a de-identification layer to all data before persistence in Redis or any external database (HIPAA, GDPR compliance)
- Audit trail: Every answer should log the query_mode, graph_intent, retrieved_chunk_ids, and reasoning_path for medical audit purposes

12.3 Handling Repeated Queries (Production Pattern)

Strategy	Mechanism	Recommended Use
Cache strategy (cost efficiency)	Semantic similarity check against prior queries via Redis VL or GPTCache; cosine similarity >0.95 returns cached answer	Use for general knowledge queries; NOT for personalised patient queries (report may have changed)
Reframing strategy (UX)	System prompt instructs LLM: if repeated question detected from conversation buffer, rephrase with simpler language or alternative analogy	Best for patients who didn't understand first answer — adds empathy without extra cost

Validation strategy (clinical)	Router detects repetition; prepends "As mentioned earlier..." prefix; ensures clinical continuity	Preferred for medical setting — acknowledges the patient, maintains professional tone

Important: Do NOT simply return a copy of the previous answer. A repeated question indicates a need for clarity, not repetition. The system should detect the repetition and adapt its explanation — a principle derived from medical communication best practices.

12.4 Monitoring - TruLens for Production

While RAGAS handles offline batch evaluation, TruLens provides the production observability layer. TruLens's RAG Triad (Context Relevance, Groundedness, Answer Relevance) complements RAGAS and enables:

- Execution-level tracing: query input → retrieval spans → reranker output → generation → latency → token usage
- Online monitoring: real-time quality scoring per query in production
- Trace-based failure analysis: identify exactly which pipeline stage caused a quality regression
- Drift detection: track score trends over time as corpus grows or LLM versions change

Recommended evaluation stack: Golden Dataset + RAGAS (offline) + TruLens (online production monitoring)

13. Future Scope

Enhancement	Rationale & Impact
Cross-encoder reranker (ms-marco-MiniLM-L-6-v2)	Post-MMR deep ranking; processes query and document together; can improve retrieval accuracy by 30–50% by catching "semantically similar but irrelevant" chunks that MMR misses. Feasible on CPU by reranking only top 10–20 candidates.
Replace DuckDuckGo with Tavily/SerpAPI	DuckDuckGo is unreliable as production web fallback — occasional blocks and empty results. Tavily or SerpAPI provide more consistent API-based search with better medical source prioritisation.
MedGemma evaluation (Google)	MedGemma 1.5 4B achieves 27.0 Macro F1 on CT-RATE (3D radiology) and 89.5 on MIMIC CXR (2D chest X-ray). As a domain-adapted VLM, it could replace the current text-based [IMAGE:] approach with actual visual understanding of clinical figures.
Text-to-Cypher with fine-tuned models	Expand beyond 6 pre-written templates. Fine-tuned or instruction-tuned models (e.g., Granite) show stronger Cypher generation accuracy. Would allow queries outside the six intent categories.
TruLens integration	Add execution-level tracing and online monitoring for the production deployment path. Complementary to existing RAGAS offline evaluation.
Synthea synthetic patient data	Synthea generates realistic synthetic patient records with diagnoses, medications, lab values, and treatment histories. Using synthetic patients as test cases would enable more realistic evaluation without privacy concerns.
MACCROBAT clinical NER dataset	MACCROBAT (Kaggle) is a medical annotation dataset for clinical NER. Could be used to train or evaluate a domain-specific NER model to replace the deterministic keyword matching in detect_query_intent().
Structured outputs for report analysis	Use Anthropic/OpenAI structured outputs API to produce JSON-formatted report analysis that can be displayed in a structured UI component rather than free-text markdown.
FHIR/HL7 integration	Standardised patient data interchange format. Integration would allow MedChat to read directly from hospital EHR systems rather than requiring PDF upload.
GPU deployment option	On GPU hardware: replace bge-base-en with a larger medical embedding model (e.g., MedCPT-Article-Encoder); replace Groq API with local LLaMA deployment for data sovereignty in clinical settings.

14. Weekly R&D Progress

This section documents the full 16-week research and development log, showing the evolution from a basic document loader experiment to a production-grade Graph RAG system. Each week is presented as a narrative that integrates decisions, learnings, and outcomes.

Week 1 — Onboarding, Architecture Alignment & Environment Setup

January 19–24. Administrative onboarding was completed on January 19. The team induction with Suraj Sir clarified the three core objectives for the internship: build a multimodal RAG system, ensure it runs on CPU-bound hardware, and make it practically useful for oncology patients.

The development environment was replicated on a personal i5 laptop on January 22, immediately surfacing the hardware constraint that would define the architecture: no GPU, <10GB RAM, Windows. A daily standup cycle with Kajal Ma'am and the intern team was initiated.

R&D focus: HIPAA and FHIR fundamentals (privacy and interoperability standards for medical data); Milvus vs FAISS comparison for vector storage; initial exploration of LangChain document loaders and RAG pipeline patterns.

Contribution: Co-designed the AI upskilling curriculum for all Aziro developers. The "Courses for various roles and levels" proposal was submitted to Priyanka Ma'am on January 21.

Week 2 — Data Ingestion Pipeline Implementation

January 27–31. The practical execution of the RAG ingestion pipeline began this week. PyMuPDF and Docling were both successfully tested on January 27 for document loading.

Research finding: Docling was initially evaluated as unsatisfactory in first tests on a Disaster Management PDF — it failed on some scanned content. However, subsequent testing on clean medical PDFs revealed it as the best option for table extraction.

Chunking research was initiated on January 30: Fixed-size (arbitrary splits at character boundaries, disrupts clinical sentences), Recursive (hierarchy of separators — recommended starting point), Document-structure-based (preserves section boundaries — critical for section_hierarchy metadata), and Semantic/Agentic (too slow on CPU). This research directly produced the two-stage MarkdownHeader + Recursive architecture.

Upskilling: 200+ external AI/ML resources were curated over two days and submitted before the deadline (January 29) as the "AI/ML Training External Links" document.

Week 3 — Multimodal Pipeline & Image Processing Experiments

February 3–7. This was the most experimentally intensive week. The objective was achieving reliable image description for the RAG pipeline without GPU infrastructure.

February 3: A lightweight CPU pipeline was implemented using Docling + PyMuPDF + Moondream/Qwen2-vl:3b. ~90 images were extracted from complex documents. AI-generated descriptions were obtained, but quality was insufficient for clinical accuracy (see the Moondream sample output in Section 8.4).

February 4: Docling's HybridChunker was integrated following guidance from Kajal Ma'am for adaptive chunking. The HybridChunker preserves structured content (code, tables, markdown) more faithfully than recursive character splitting alone.

February 5: Workflow developed to store AI-generated image descriptions as text in embedding-ready format. ~20 images were successfully embedded using embeddinggemma, creating a prototype unified multimodal vector space.

Upskilling launch: The first "Upskilling Training" meeting was held February 2. The Hiring Automation Tool demo was given by Ravi Sir — a Gemini API-driven system with automated Google Form creation and bulk emailing capabilities.

Weeks 4–5 — Retrieval Innovation: Hybrid Search & MMR

February 10–21. The RAG Search Lab was built this week — a dashboard for comparing retrieval strategies quantitatively. This produced the comparative tables in Section 3.2.

February 10 (Cross-encoder): A cross-encoder reranker was evaluated. Key insight from this experiment: bi-encoders (like bge-base-en) produce "vector vs vector" comparisons that lose nuance; cross-encoders process the query and document together using attention across tokens. Even on an i5, reranking only the top 10–20 candidates (rather than the full index) is computationally feasible and can improve accuracy by 30–50%. This evaluation informed the cross-encoder being listed as the top future scope item.

February 11 (MMR): Maximal Marginal Relevance was tested. The critical learning: standard semantic search returns 5 chunks that all say the same thing, wasting the context window. MMR's lambda parameter was tuned: $\lambda=0.5$ was too diverse (clinical coherence lost); $\lambda=0.7$ produced near-duplicate clinical paragraphs; $\lambda=0.6$ produced the right balance for medical text.

Score normalisation: BM25 scores (0–10+) and vector scores (0.0–1.0) use incompatible scales. RRF was implemented as the normalisation layer — it converts both to rank-based scores using $1/(k+rank+1)$, ensuring neither strategy dominates. This was a critical insight from the RAG Search Lab experiments.

Week 6 — Graph Database Integration & Three-Tier Architecture

This was the architectural pivot week. The Three-Tier Architecture was defined following the pipeline established in Tuesday's planning session:

- Ingestion Tier: Processes Figshare Radiograph dataset and TCGA-SARC MRI metadata
- Embedding Tier: Uses OpenAI Vision (initial plan) to interpret visual features like the sunburst periosteal reaction in Osteosarcoma X-rays and convert them into vectors
- Knowledge Tier: Maps vectors to Neo4j graph where "Osteosarcoma" nodes are linked to "Complications" such as Avascular Necrosis (AVN) or Osteonecrosis, allowing the RAG to explain why certain treatments cause bone death

The patients.csv dataset provided ground truth for AJCC/Enneking stages and MAP chemotherapy treatment plans. The reports_index.csv acted as the lookup table linking specific MRI/CT text files to patient IDs. Key clinical insight derived: different osteosarcoma subtypes (Telangiectatic, Small Cell, Parosteal) have significantly different tumor sizes (3.4cm to 21cm+) and require unique embedding strategies. High prevalence in Distal Femur and Proximal Tibia — key regions for monitoring AVN following intensive chemotherapy.

Week 7 — Model Evaluation & Comparative Analysis

MedGemma was evaluated as a potential alternative to LLaMA-3 for the generation component. Results from the Google Cloud Vertex AI model garden:

Task/Dataset	Gemma 3 4B	MedGemma 1 4B	MedGemma 1.5 4B	MedGemma 1 27B
CT Dataset 1 (Macro Acc.)	54.5	58.2	61.1	57.8
CT-RATE (Macro F1)	—	23.5	27.0	—
CT-RATE (Precision)	—	34.5	34.2	—
CT-RATE (Recall)	—	34.1	42.0	—
MRI Dataset 1 (Macro Acc.)	51.1	51.3	64.7	57.4
MIMIC CXR (F1 top-5)	81.2	88.9	89.5	90.0

Decision: Groq LLaMA-3.3-70b was retained for generation (text-based answers). MedGemma is listed as future scope for VLM-based visual understanding of clinical figures — particularly relevant for interpreting the radiology images that the current [IMAGE:] tag approach only references without understanding.

Week 8 — RAGAS Evaluation Framework

The RAGAS evaluation framework was integrated this week. Initial challenge: evaluating without a golden dataset produced uninterpretable scores — no baseline, no failure attribution.

A structured golden dataset creation workflow was introduced with four query categories: simple factual queries, multi-hop reasoning queries, graph relation queries, and retrieval stress-test queries. This evolved into the 20-question test set described in cancer_evaluation.py.

Critical discovery: the vibrantlabsai fork of ragas (version 0.4.x) was installed by default on this system and silently broke evaluate() at runtime. The Faithfulness class does not inherit from Metric, causing "All metrics must be initialised metric objects" with no indication that a wrong package is installed. Solution: pip uninstall ragas -y && pip install "ragas==0.2.6" from the official explodinggradients package.

LangChain EnsembleRetriever was evaluated as an alternative to the custom RRF fusion. While it provides a BM25 + semantic combination natively, it uses simple weighted sum combination rather than rank-based fusion — making it more susceptible to score scale differences. The custom RRF implementation was retained.

Weeks 9–11 — Three-Mode System (v5)

The pipeline evolved from the v4 LLMGraphTransformer-based enrichment to the v5 three-mode system. Key changes:

- Removed: LLMGraphTransformer, spaCy entity extraction, QueryRouter (local/global), GraphEnricher class, Community node queries
- Added: Three-mode routing (RESEARCH/GRAPH/AUTO); GraphRetriever class with six pre-written Cypher templates; deterministic detect_query_intent() entity matching
- Preserved: BM25 + dense + RRF + MMR pipeline; [IMAGE:] tag prompt instruction; web fallback; streaming; follow-up generation

The transition from LLMGraphTransformer to hand-crafted Cypher was driven by quality concerns: auto-extracted graphs had inconsistent node types and missed critical nutrition relationships. The hand-crafted schema took longer but produced verifiable, auditable clinical relationships that a domain expert can check.

Weeks 12–13 — v5.1 Bug-Fix Release & Final Evaluation

The five critical fixes implemented in v5.1 addressed the most impactful quality issues discovered during RAGAS evaluation runs:

Fix	Root Cause & Solution

Fix 1: Image chunks dropped by MMR	Root cause: image chunks (short, descriptive) always lost to prose chunks (long, detailed) in shared MMR. Solution: dedicated image BM25 index; chunks injected directly into ## Visual References section bypassing MMR entirely.
Fix 2: LLM skipping image tags	Root cause: permissive language ("if context contains images, include them") gave LLM the option to skip. Solution: mandatory "MUST include", "ALWAYS reference", dedicated VISUAL REFERENCES INSTRUCTIONS block above the answer section.
Fix 3: Web fallback trigger too strict	Root cause: exact phrase matching missed ~40% of inadequate answers where "I don't know" was buried in paragraphs. Solution: three-tier detection (short answer + any phrase; long answer + multiple phrases; 5 regex patterns for partial admissions).
Fix 4: Confusing fallback presentation	Root cause: failed RAG answer prepended before web result created "I don't know... but here's the answer" confusion. Solution: web answer standalone when RAG empty; labelled "Additional sources" when RAG partial.
Fix 5: DuckDuckGo query quality	Root cause: "cancer oncology" suffix added noise for specific regulatory queries. Solution: suffix only added when query doesn't already contain medical terms.

Weeks 14-15 - v5.1 Evaluation Using Ragas and various other custom metrics

Week 14 focused on refining the golden dataset — ground truths were shortened where necessary.. N-gram metrics (BLEU, ROUGE-L) were also explored as a supplementary lexical signal alongside RAGAS; while cheaper to run and useful for rapid regression checks, they penalise paraphrasing and are best treated as a directional indicator rather than an absolute quality measure.

Week 15 was dominated by the automatic suspension of the Neo4j AuraDB free-tier instance, which pauses after 72 hours of inactivity. Restoring the pipeline required manually resuming the instance, re-running ingestion to rebuild the vector index, and re-executing graph builder scripts to reconstruct the knowledge graph — a process taking approximately two-four hours. Following restoration, debugging was conducted on RAGAS edge cases where faithfulness scores were unexpectedly low, with several traced to the LLM judge misinterpreting paraphrased citations as hallucinations. The week concluded with consolidating the full R&D log and architectural decisions into this document.

15. Conclusion

MedChat demonstrates that a clinically meaningful cancer assistant is achievable on consumer hardware with zero GPU infrastructure. Over 13 weeks, the system evolved from a simple PDF loader experiment into a production-grade Graph RAG architecture combining three retrieval

strategies, a hand-crafted medical knowledge graph, dedicated visual retrieval, and a robust fallback system.

The most important architectural insight was the necessity of explicit retrieval routing. Attempting to answer all clinical queries with a single retrieval strategy — whether BM25-only, semantic-only, or graph-only — produces a system that is mediocre at everything. The three-mode design (RESEARCH for clinical literature, GRAPH for drug–food interactions, AUTO for combined) allows each component to excel within its domain while remaining transparent to the user.

The most practically impactful engineering decision was the image retrieval fix: separating image chunk retrieval from content chunk retrieval via a dedicated BM25 index. This one architectural change converted image questions from 0% [IMAGE:] tag rate to 100%, making the system genuinely multimodal rather than just text-with-image-mentions.

The most underappreciated lesson was around evaluation infrastructure. Implementing RAGAS from Week 8 with a carefully designed golden dataset was what made all subsequent debugging systematic rather than empirical. Without RAGAS, the image retrieval bug might have been attributed to the wrong component for weeks.

The remaining gap between this POC and production deployment is primarily operational (reliability of the DuckDuckGo fallback, PII compliance for patient data, TruLens online monitoring) and capability (cross-encoder reranking for higher precision, MedGemma for visual understanding). Both are achievable incremental improvements on the current foundation.

Dimension	Achievement	Gap	Next Step
Retrieval Quality	Hybrid BM25+Dense+RRF+MMR; three-mode routing	No cross-encoder reranker yet	Add ms-marco-MiniLM cross-encoder post-MMR
Multimodal	[IMAGE:] tag injection; dedicated image index	No visual understanding of figures	MedGemma evaluation for VLM captioning
Graph Reasoning	Six Cypher templates; hand-crafted schema	Limited to six query patterns	Text-to-Cypher with fine-tuned model
Evaluation	RAGAS + custom metrics + HTML dashboard + per-run history	No online production monitoring	TruLens integration
Production Safety	Out-of-corpus detection; mandatory disclaimers; context-only prompts	No PII stripping; no audit trail	De-identification pipeline + structured logging

16. References

Primary Tools & Libraries

- LangChain EnsembleRetriever: https://reference.langchain.com/python/langchain-classic/retrievers/ensemble/EnsembleRetriever
- Evaluating Medical RAG with NVIDIA AI Endpoints and Ragas: https://developer.nvidia.com/blog/evaluating-medical-rag-with-nvidia-ai-endpoints-and-ragas/
- MACCROBAT Medical Annotation Dataset: https://www.kaggle.com/datasets/okolojeremiah/maccrobat/data
- MedGemma Model Garden: https://console.cloud.google.com/vertex-ai/publishers/google/model-garden/medgemma
- MedGemma 1.5 4B (HuggingFace): https://huggingface.co/google/medgemma-1.5-4b-it
- Synthea Synthetic Patient Generator: https://syntheticheath.github.io/synthea/
- Docling Pipeline (YouTube): https://www.youtube.com/watch?v=YAgjtZKLVKo
- LangChain Text Splitters Documentation: https://docs.langchain.com/oss/python/integrations/splitters
- Chunking Strategies Overview (YouTube): https://www.youtube.com/watch?v=8OJC21T2SL4
- Interactive Chunking Visualisation: https://chunkviz.up.railway.app/

Architecture & Strategy References

- OpenAI Structured Outputs: https://platform.openai.com/docs/guides/structured-outputs
- IBM Granite Vision 3.2-2B: https://huggingface.co/ibm-granite/granite-vision-3.2-2b
- IBM Developer: Enhancing RAG with Chunking Strategies:https://developer.ibm.com/articles/awb-enhancing-rag-performance-chunking-strategies/
- DataCamp: Multimodal RAG Tutorial: https://www.datacamp.com/tutorial/multimodal-rag
- NVIDIA: Introduction to Multimodal RAG: https://developer.nvidia.com/blog/an-easy-introduction-to-multimodal-retrieval-augmented-generation/
- Qdrant FastEmbed: https://github.com/qdrant/fastembed
- Qdrant Embeddings Search (OpenAI Cookbook): https://developers.openai.com/cookbook/examples/vector_databases/qdrant/using_qdrant_for_embeddings_search

Setup & Configuration Guides

- Neo4j AuraDB Quick Start Guide — *Neo4j_AuraDB_Quick_Start_Guide.docx* — Step-by-step setup for Neo4j Aura free-tier instances, including instance provisioning, credential download, Cypher basics, free-tier pros and cons, instance-pausing behaviour, and the Graph RAG reconnection pattern.

- Groq API Quick Start Guide — *GroqAPI_QuickGuide.pdf* — Registration, API key creation, model selection, Python SDK usage, and free-tier rate-limit reference for the Groq inference platform.

MedChat — Graph RAG Cancer Assistant

POC Technical Documentation v1.0 | Aziro Technologies | Internal Use Only